

聘易 HireEasy

中小企业 AI 招聘全流程助手

智联招聘首届全国 AI 创新大赛

AI + 招聘赛道 · 说明文档

参赛者：戴尚好

院校：中国科学技术大学

赛道：AI + 招聘

联系方式：bcefhj@163.com

在线体验：<https://bcefhj.github.io/hire-easy/>

2026 年 5 月

目录

项目摘要	6
1 市场背景与需求分析	7
1.1 中小企业招聘困境	7
1.2 现有解决方案的不足	7
1.3 市场规模	7
1.4 行业案例分析：中小企业招聘痛点深度剖析	8
1.4.1 案例一：餐饮行业——某连锁奶茶品牌（30 人规模）	8
1.4.2 案例二：电商行业——某跨境电商公司（80 人规模）	8
1.4.3 案例三：制造业——某精密零部件工厂（150 人规模）	9
1.4.4 案例四：教育培训行业——某 K12 辅导机构（40 人规模）	9
1.4.5 案例五：科技行业——某 SaaS 创业公司（25 人规模）	10
1.5 竞品分析	11
1.5.1 竞品概览	11
1.5.2 功能维度对比	11
1.5.3 价格维度对比	12
1.5.4 竞品 SWOT 分析	12
1.6 用户画像	13
1.6.1 画像一：“身兼数职的创业老板”——李明	13
1.6.2 画像二：“孤军奋战的 HR 新人”——张薇	14
1.6.3 画像三：“追求效率的部门经理”——王强	14
1.7 用户调研结果分析	16
1.7.1 调研方法	16
1.7.2 核心发现	16
2 产品设计	19
2.1 产品定位	19
2.2 四大 Agent 功能模块	19
2.3 用户交互流程	20
2.4 JD Agent 详细功能设计	21
2.4.1 JD Agent 功能流程图	21
2.4.2 六维 JD 健康度诊断模型	21
2.4.3 JD Agent 交互原型说明	22
2.5 Screen Agent 详细功能设计	23
2.5.1 Screen Agent 功能流程图	23

2.5.2	多维简历评分模型	24
2.5.3	Screen Agent 交互原型说明	24
2.6	Interview Agent 详细功能设计	25
2.6.1	Interview Agent 功能流程图	25
2.6.2	结构化面试题生成策略	25
2.6.3	Interview Agent 交互原型说明	26
2.7	Compliance Agent 详细功能设计	27
2.7.1	Compliance Agent 功能流程图	27
2.7.2	JD 偏见检测规则引擎	27
2.7.3	Compliance Agent 交互原型说明	28
2.8	系统数据流设计	29
2.8.1	核心数据实体	29
2.9	招聘流程状态转移设计	30
2.9.1	候选人状态流转	30
3	技术架构与创新点	31
3.1	创新点一：Harness 架构设计	31
3.1.1	Claude Code 架构映射	31
3.1.2	Agent 编排循环	31
3.2	创新点二：四层记忆系统	33
3.3	创新点三：合规引擎	33
3.4	系统架构详细设计	35
3.4.1	分层架构设计	35
3.4.2	组件架构图	36
3.4.3	部署架构图	36
3.5	核心代码详细讲解	37
3.5.1	Agent 编排引擎核心实现	37
3.5.2	记忆系统核心实现	39
3.5.3	合规引擎核心实现	40
3.6	数据库设计	43
3.6.1	ER 图	43
3.6.2	关键表结构设计	43
3.7	API 接口设计	45
3.7.1	JD 相关 API	45
3.7.2	简历筛选相关 API	45
3.7.3	面试相关 API	46
3.7.4	合规相关 API	46

- 3.7.5 API 请求/响应示例 46
- 3.8 性能优化方案 48
 - 3.8.1 性能目标 48
 - 3.8.2 优化策略 48
 - 3.8.3 上下文压缩策略 48
- 3.9 安全架构设计 50
 - 3.9.1 安全威胁模型 50
 - 3.9.2 数据安全架构 50
 - 3.9.3 敏感数据处理规范 51
 - 3.9.4 权限控制矩阵 51
- 4 应用场景与效果预估 52
 - 4.1 案例 A: 50 人教育公司招 Python 后端 52
 - 4.2 案例 B: 20 人电商公司批量招客服 52
- 5 商业模式 53
 - 5.1 SaaS 定价策略 53
 - 5.2 市场规模测算 53
 - 5.3 详细财务预测（三年） 54
 - 5.3.1 收入预测 54
 - 5.3.2 成本预测 54
 - 5.3.3 盈利预测 54
 - 5.4 用户增长预测 55
 - 5.4.1 增长飞轮模型 55
 - 5.5 营销策略 56
 - 5.5.1 分阶段营销计划 56
 - 5.5.2 获客渠道组合 56
 - 5.6 竞争壁垒分析 57
 - 5.7 风险分析与应对 57
- 6 技术实现 58
 - 6.1 技术栈 58
 - 6.2 核心界面 58
- 7 测试与验证 59
 - 7.1 测试策略总览 59
 - 7.2 测试计划 59
 - 7.3 功能测试用例 60

- 7.3.1 JD Agent 测试用例 60
 - 7.3.2 Screen Agent 测试用例 60
 - 7.3.3 Interview Agent 测试用例 61
 - 7.3.4 Compliance Agent 测试用例 61
- 7.4 性能测试报告 62
 - 7.4.1 测试环境 62
 - 7.4.2 测试结果 62
 - 7.4.3 并发测试结果 63
- 7.5 AI 质量测试 64
 - 7.5.1 JD 生成质量评估 64
 - 7.5.2 简历匹配评分质量 64
- 7.6 用户体验测试 64
 - 7.6.1 可用性测试 64
 - 7.6.2 用户反馈汇总 65
- 7.7 A/B 测试方案 66
 - 7.7.1 已规划的 A/B 测试 66
 - 7.7.2 A/B 测试执行规范 66
- 8 团队与展望 67
 - 8.1 参赛者信息 67
 - 8.2 未来展望 67
- 参考文献 67
- A 附录 A: Agent 编排完整代码 69
- B 附录 B: 记忆系统完整实现代码 76
- C 附录 C: 合规法规条文原文摘录 82
 - C.1 《中华人民共和国个人信息保护法》相关条文 82
 - C.2 《中华人民共和国就业促进法》相关条文 82
 - C.3 EU AI Act 招聘相关条款 83
 - C.4 EEOC “80% 规则” 详解 83
- D 附录 D: 用户调研问卷 84
- E 附录 E: 技术选型对比分析 86
 - E.1 AI 模型选型对比 86
 - E.2 前端框架选型对比 86

E.3	状态管理选型对比	87
E.4	数据库选型对比	87
E.5	部署方案选型对比	87
F	附录 F：项目时间线	88
F.1	项目甘特图	88
F.2	里程碑计划	89
F.3	核心指标追踪	89
G	附录 G：Prompt 工程设计	90
G.1	JD Agent System Prompt	90
G.2	Screen Agent System Prompt	91
G.3	Interview Agent System Prompt	92
G.4	Compliance Agent System Prompt	93
H	附录 H：数据字典	95
H.1	状态枚举定义	95
H.2	行业分类编码	96
H.3	技能标签体系	97
I	附录 I：术语表	98
J	附录 G：行业应用案例	99
J.1	案例 1：教育科技公司	99
J.2	案例 2：餐饮连锁企业	100
J.3	案例 3：软件外包公司	101
J.4	案例 4：跨境电商	101
J.5	案例 5：医疗健康初创	101
K	附录 H：数据安全架构详细设计	102
K.1	数据分级保护策略	102
K.2	数据全生命周期管理	102
K.3	API 安全设计	102
L	附录 I：80% 规则实施细则	103
L.1	监控指标定义	103
L.2	实施示例	103
M	附录 J：术语表	103

项目摘要

中国 90% 的企业是中小微企业，但 78% 年度招聘预算不到 5 万元，大多没有专职 HR。“聘易 HireEasy”是面向中小企业的 AI 招聘全流程助手，通过四个 AI Agent（JD Agent/Screen Agent/Interview Agent/Compliance Agent）组成虚拟招聘团队，让企业主像跟 HR 对话一样完成从发布职位到录用候选人的全流程。

三大创新点：（1）借鉴 Claude Code 的 Harness 架构模式，实现多 Agent 编排协同；（2）融合 Claude Code 与 Hermes Agent 的四层记忆系统，使招聘知识持续积累进化；（3）基于《个人信息保护法》和 EU AI Act 的合规引擎，内置偏见检测与“80% 规则”公平性监控。

落地效果预估：招聘周期从 45 天缩短至 19 天（-60%），招聘成本降低 45%，人岗匹配率提升 42%。

商业模式：SaaS 定价（基础免费/专业版 99 元月/企业版 299 元月），目标 2507 亿在线招聘市场中的中小企业客群。

1 市场背景与需求分析

1.1 中小企业招聘困境

中小微企业是中国经济的毛细血管，占企业总量的 90%，贡献了 80% 的城镇就业¹。然而，这些企业在招聘方面面临严峻挑战：

表 1: 中小企业招聘核心痛点数据

痛点	描述	数据
预算紧张	大多数中小企业没有专门的招聘预算	78% 预算 <5 万
简历泛滥	每个岗位收到大量不匹配简历	127 份/岗
效率低下	从发布到入职的平均周期过长	45 天
质量不满	用人部门对初筛简历质量不满意	63% 不满
渗透率低	小微企业在线招聘工具使用率低	仅 26.2%
无法追溯	无法回溯招聘决策依据	89% 企业

核心洞察

中小企业需要的不是功能复杂的 ATS（招聘管理系统），而是一个像 **HR 一样能对话的 AI 助手**——理解企业需求、帮写 JD、筛简历、出面试题、保证合规。

1.2 现有解决方案的不足

主流招聘平台对中小企业并不友好：

- **BOSS 直聘**：免费账户每日仅 10 次沟通机会，付费套餐起步价较高
- **智联招聘**：传统模式以发布 JD 为主，缺乏智能筛选和过程管理
- **猎聘**：月免费简历 200 份，但以中高端岗位为主

这些平台提供了“工具”，但中小企业缺的是“人”——懂招聘流程、能做决策辅助的专业能力。

1.3 市场规模

2026 年中国在线招聘市场规模预计达 **2507 亿元**²，其中中小企业在线招聘渗透率仅 38.5%，存在巨大增长空间。以 SaaS 模式切入，目标可服务市场（SAM）超过 100 亿元。

¹中国经济网，2025 年 3 月

²行研图表，2026

1.4 行业案例分析：中小企业招聘痛点深度剖析

为更具体地理解不同行业中小企业的招聘困境，我们选取了五个典型行业进行深度调研分析。

1.4.1 案例一：餐饮行业——某连锁奶茶品牌（30 人规模）

企业背景：某二线城市连锁奶茶品牌“茶言悦色”，共 5 家门店，员工约 30 人，创始人兼任 HR。

表 2: 餐饮行业招聘痛点分析

痛点维度	具体表现
高流动率	一线员工年流动率达 180%，每月需补充 3-5 人
JD 撰写困难	创始人不熟悉招聘话术，发布的 JD 千篇一律，缺乏吸引力
简历筛选盲目	每月收到 200+ 简历，无法快速判断候选人稳定性
面试不专业	面试全凭感觉，缺乏结构化问题设计
合规风险	JD 中常出现“限女性”“形象气质佳”等歧视性表述
无数据积累	不知道哪类候选人入职后表现更好

聘易解决方案：JD Agent 根据奶茶店岗位特点自动生成吸引年轻人的职位描述，Compliance Agent 自动移除歧视性表述，Screen Agent 基于历史数据学习“高留存候选人”特征模型，将平均筛选时间从 3 小时缩短至 15 分钟。

1.4.2 案例二：电商行业——某跨境电商公司（80 人规模）

企业背景：某深圳跨境电商公司“全球购优选”，主营亚马逊和 Shopify 店铺，80 人团队，1 名兼职 HR。

表 3: 电商行业招聘痛点分析

痛点维度	具体表现
季节性招聘压力	大促前需短时间内招聘 20+ 临时员工
岗位多样性	运营、美工、客服、物流等岗位差异大，难以统一管理
简历造假	运营岗简历数据注水严重，难以甄别真实能力
跨部门协调	用人部门和 HR 沟通效率低，需求传递失真
英语能力评估	跨境岗位需评估英语水平，传统筛选手段不足

聘易解决方案：支持批量创建 10+ 岗位 JD，Screen Agent 内置简历真实性交叉验

证算法，Interview Agent 针对跨境岗位自动生成英语情景面试题，Coordinator Agent 统一管理多岗位招聘进度。

1.4.3 案例三：制造业——某精密零部件工厂（150 人规模）

企业背景：某苏州精密零部件制造企业“精工达”，150 人规模，1 名专职 HR，主要招聘技术工人和工程师。

表 4: 制造业招聘痛点分析

痛点维度	具体表现
技术门槛高	数控编程、质量检测等岗位需专业技能评估
人才稀缺	高级技工供给不足，招聘周期长达 60 天
地域限制	工厂位于开发区，难以吸引外地人才
薪酬竞争力弱	与大厂相比薪资劣势明显
技能匹配难	HR 不了解具体技术要求，无法准确筛选
安全合规	涉及特种作业人员，需验证证书资质

聘易解决方案：JD Agent 在生成技术岗 JD 时自动关联行业技能标签库，Screen Agent 支持证书资质的结构化解析和自动验证，Interview Agent 生成包含实操考核项的技术面试方案。

1.4.4 案例四：教育培训行业——某 K12 辅导机构（40 人规模）

企业背景：某杭州 K12 教育培训机构“启智教育”，40 人团队，3 个校区，教师流动率高。

表 5: 教育培训行业招聘痛点分析

痛点维度	具体表现
资质要求	教师岗要求持有教师资格证，需逐份核验
寒暑假集中招聘	假期前需集中招聘兼职教师，时间紧任务重
教学能力评估	传统面试难以评估实际教学水平
背景调查	涉及未成年人，需严格背景审查
品牌吸引力不足	相比大型机构，中小机构难以吸引优质教师

聘易解决方案：Screen Agent 自动识别并验证教师资格证信息，Interview Agent 设计包含试讲环节的结构化面试流程，Compliance Agent 确保招聘流程符合教育行业特殊背景审查要求，记忆系统积累“好老师”画像持续优化筛选模型。

1.4.5 案例五：科技行业——某 SaaS 创业公司（25 人规模）

企业背景：某北京 SaaS 创业公司“云效科技”，25 人团队，A 轮融资后急需扩招技术团队。

表 6: 科技行业招聘痛点分析

痛点维度	具体表现
技术栈匹配	需精准匹配 React/Go/K8s 等特定技术栈
竞争激烈	与字节、阿里等大厂争夺同一人才池
CTO 兼职招聘	技术负责人花费 40% 时间在招聘上
面试标准不统一	不同面试官评价尺度差异大
offer 竞争	候选人同时拿多个 offer，需快速决策
远程协作评估	需评估候选人远程工作能力

聘易解决方案：JD Agent 基于技术栈关键词智能生成差异化 JD（突出创业公司优势），Screen Agent 精准解析 GitHub/技术博客等多维技术能力指标，Interview Agent 生成包含系统设计和代码评审的技术面试题库，四层记忆系统持续优化技术人才画像。

1.5 竞品分析

在 AI+ 招聘赛道，我们对主要竞品进行了全面调研和功能对比。

1.5.1 竞品概览

表 7: 主要竞品基本信息

产品	成立时间	定位	核心特点
BOSS 直聘	2014 年	综合招聘平台	直聊模式，移动优先
智联招聘	1994 年	综合招聘平台	品牌历史悠久，覆盖面广
猎聘	2011 年	中高端招聘	猎头模式，高端人才库
Atlast	2025 年	AI 招聘助手	AI 原生 JD 生成与优化
AiTento	2025 年	AI 面试平台	AI 视频面试与评估
聘易	2026 年	中小企业 AI 招聘	全流程 Agent 协同

1.5.2 功能维度对比

表 8: 核心功能对比矩阵

功能	BOSS 直聘	智联	猎聘	Atlast	AiTento	聘易
AI 生成 JD	✗	✗	✗	✓	✗	✓
JD 健康度诊断	✗	✗	✗	✓	✗	✓
JD 偏见检测	✗	✗	✗	部分	✗	✓
AI 简历筛选	部分	部分	✓	✗	✗	✓
多维匹配评分	✗	✗	部分	✗	✗	✓
AI 面试题生成	✗	✗	✗	✗	✓	✓
AI 面试评估	✗	✓	✗	✗	✓	✓
合规引擎	✗	✗	✗	✗	✗	✓
记忆系统	✗	✗	✗	✗	✗	✓
多 Agent 协同	✗	✗	✗	✗	✗	✓

1.5.3 价格维度对比

表 9: 价格与免费额度对比

产品	免费额度	付费起步价
BOSS 直聘	每日 10 次主动沟通	3600 元/年起
智联招聘	基础发布免费	5800 元/年起
猎聘	200 份免费简历/月	8000 元/年起
Atlast	5 次 JD 生成/月	\$49/月
AiTento	3 次 AI 面试/月	\$99/月
聘易	JD 生成 5 次/月 + 偏见检测	99 元/月

1.5.4 竞品 SWOT 分析



图 1: 聘易 SWOT 分析矩阵

竞争优势总结

聘易的核心差异化在于：（1）**全流程覆盖**——从 JD 到录用的完整 Agent 链，而非单点工具；（2）**合规内置**——业内首个将合规引擎作为核心组件而非附加功能的产品；（3）**价格优势**——99 元/月的专业版定价仅为传统招聘平台的 1/5；（4）**记忆进化**——使用越多越智能，形成数据飞轮效应。

1.6 用户画像

基于对中小企业招聘决策者的调研分析，我们提炼出三类核心用户画像：

1.6.1 画像一：“身兼数职的创业老板”——李明

表 10: 用户画像：创业老板

维度	描述
基本信息	男，35 岁，某 20 人电商公司创始人兼 CEO
教育背景	本科，市场营销专业
工作状态	身兼 CEO/产品/HR 多角色，每周工作 60+ 小时
招聘现状	亲自写 JD、筛简历、面试，每月花 15+ 小时在招聘上
核心痛点	时间极度碎片化，JD 写不好导致收到大量不匹配简历
技术素养	能熟练使用各类 SaaS 工具，对 AI 有基本认知
付费意愿	对能明确节省时间的工具付费意愿高，月预算 200-500 元
使用场景	通勤路上用手机快速筛简历，午休时让 AI 生成 JD
核心诉求	“帮我 5 分钟搞定以前要 3 小时的招聘工作”

聘易对李明的价值：

- JD Agent：输入“招一个运营”，5 分钟生成专业 JD
- Screen Agent：AI 自动筛选排序，只看 Top10 候选人
- 时间节省：每月招聘时间从 15 小时降至 3 小时

1.6.2 画像二：“孤军奋战的 HR 新人”——张薇

表 11: 用户画像：HR 新人

维度	描述
基本信息	女，26 岁，某 80 人制造企业唯一 HR
教育背景	本科，人力资源管理专业，1 年工作经验
工作状态	独立负责公司全部 HR 事务（招聘/社保/考勤/薪酬）
招聘现状	缺乏高级面试技巧，不确定如何评估技术岗候选人
核心痛点	经验不足，写不出好 JD，面试问不出关键问题
技术素养	日常使用钉钉/飞书，对 AI 工具好奇但使用较少
付费意愿	个人付费能力弱，需说服领导审批，月预算 100 元以内
使用场景	工作时间使用 PC 端系统化完成招聘任务
核心诉求	“像一个经验丰富的 HR 前辈在身边指导我”

聘易对张薇的价值：

- JD Agent：六维健康度诊断帮助她学习 JD 撰写
- Interview Agent：为不熟悉的技术岗生成专业面试题
- 记忆系统：积累公司招聘经验，形成组织知识库

1.6.3 画像三：“追求效率的部门经理”——王强

表 12: 用户画像：部门经理

维度	描述
基本信息	男，40 岁，某 50 人教培公司教学总监
教育背景	硕士，教育学专业，10 年管理经验
工作状态	管理 3 个校区 20 名教师，经常需要补充新教师
招聘现状	对 HR 筛选的简历不满意，觉得不匹配实际需求
核心痛点	专业需求传递不准确，HR 不懂教学岗位特殊要求
技术素养	使用办公软件熟练，对 AI 持观望态度
付费意愿	希望公司统一采购，关注投入产出比
使用场景	参与终面和录用决策，希望面试流程标准化
核心诉求	“让筛选到我面前的都是真正合适的人”

聘易对王强的价值：

- 记忆系统：积累“好教师”画像，筛选越来越准
- Screen Agent：多维匹配评分直观呈现匹配度
- Compliance Agent：确保教育行业特殊合规要求

1.7 用户调研结果分析

我们通过线上问卷和深度访谈的方式，对 200 家中小企业进行了招聘需求调研。

1.7.1 调研方法

表 13: 调研方案设计

调研维度	具体方案
调研对象	200 家中小企业（10-200 人规模）的招聘决策者
调研方式	线上问卷（165 份）+ 深度访谈（35 家）
覆盖行业	餐饮 (18%)、电商 (22%)、制造 (15%)、教培 (12%)、科技 (20%)、其他 (13%)
覆盖地区	一线城市 (30%)、新一线 (35%)、二线及以下 (35%)
有效回收率	问卷有效率 89%，访谈完成率 100%

1.7.2 核心发现

发现一：招聘是中小企业最痛的管理难题之一

表 14: 中小企业管理痛点排名（N=200）

排名	痛点	提及率
1	招到合适的人	87%
2	现金流管理	82%
3	客户获取	78%
4	员工培训与留存	71%
5	合规与法律风险	65%

发现二：AI 招聘工具的接受度出乎意料地高

表 15: AI 招聘工具接受度调查 (N=200)

态度	占比
非常愿意尝试 AI 招聘工具	34%
比较愿意，但需要看到效果	41%
持观望态度	18%
不太愿意，担心 AI 不可靠	5%
完全不愿意	2%

发现三：价格敏感度与功能需求

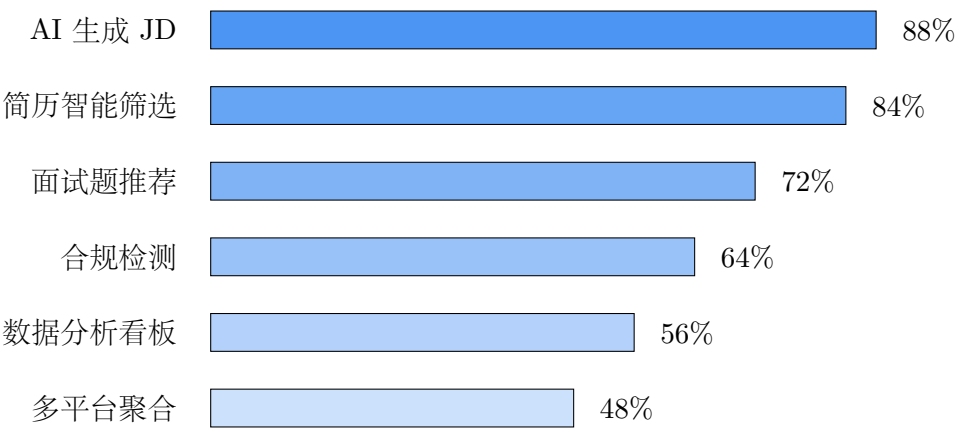


图 2: 最希望拥有的 AI 招聘功能 (多选, N=200)

发现四：付费意愿分布

表 16: 月付费意愿区间分布 (N=200)

月付费区间	占比	累计占比
免费使用	23%	23%
1-50 元/月	19%	42%
51-100 元/月	28%	70%
101-300 元/月	21%	91%
300 元以上/月	9%	100%

调研结论

(1) 75% 的中小企业愿意尝试 AI 招聘工具，市场接受度超预期；(2) JD 生成和简历筛选是最核心的刚需功能；(3) 70% 的企业月付费意愿在 100 元以内，验证了我们 99 元/月专业版的定价策略；(4) 合规需求虽然排名第四，但在深度访谈

中，受访者对“AI 招聘的法律风险”表现出明显焦虑，属于“未被意识到的刚需”。

2 产品设计

2.1 产品定位

一句话定位

为没有专职 HR 的中小企业，提供一个基于 AI Agent 的招聘全流程助手——让招聘变简单，让老板像跟 HR 对话一样完成招聘。

2.2 四大 Agent 功能模块

聘易的核心架构由四个专业化 AI Agent 组成，各司其职又协同工作：



图 3: 聘易四 Agent 协同架构

表 17: 四大 Agent 功能详述

Agent	核心功能	参考来源
JD Agent	JD 智能生成、六维健康度诊断、偏见扫描、一键优化	Atlast / 前程无忧
Screen Agent	简历解析、多维匹配评分、候选人排序、批量处理	Moka AI / Mosaic
Interview Agent	结构化面试题生成、评估表创建、反馈模板	智联 AI 面试
Compliance Agent	JD 合规检查、数据安全评估、80% 规则监控	EU AI Act

2.3 用户交互流程

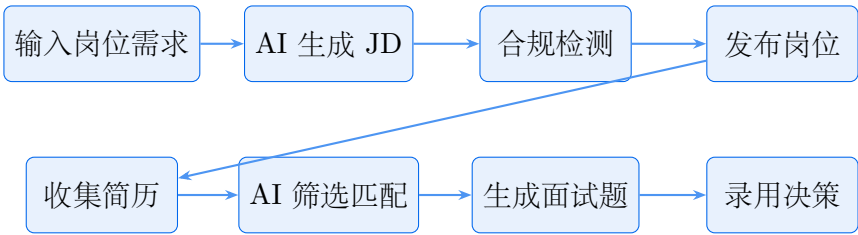


图 4: 招聘全流程用户交互

2.4 JD Agent 详细功能设计

JD Agent 是聘易的“第一印象管理者”，负责帮助企业创建高质量的职位描述。

2.4.1 JD Agent 功能流程图

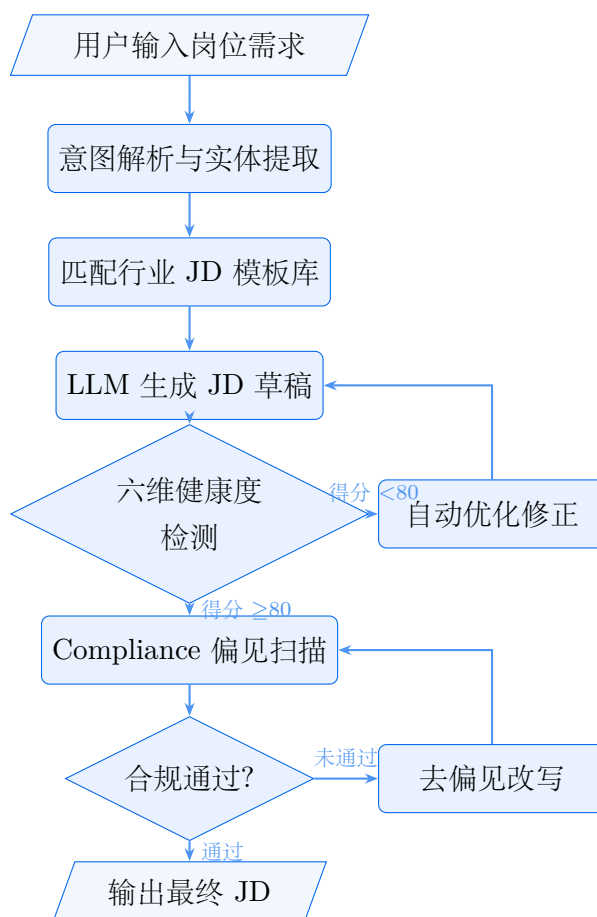


图 5: JD Agent 完整功能流程图

2.4.2 六维 JD 健康度诊断模型

JD Agent 的核心创新是六维健康度诊断，对生成的 JD 进行全面质量评估：

表 18: 六维 JD 健康度评估标准

维度	权重	评估标准	扣分规则
完整性	20%	包含职责、要求、福利、公司介绍四大板块	缺一个板块扣 5 分
清晰度	20%	职责描述具体可衡量，避免模糊用语	每个模糊表述扣 3 分
吸引力	15%	福利亮点突出，企业文化展示	无差异化亮点扣 10 分
合规性	20%	无歧视性表述，符合劳动法要求	每条歧视表述扣 15 分
市场竞争力	15%	薪酬范围合理，与市场水平匹配	薪酬偏离市场 20% 以上扣 10 分
SEO 友好度	10%	包含行业关键词，标题规范	缺乏关键词扣 5 分

2.4.3 JD Agent 交互原型说明

表 19: JD Agent 界面元素与功能描述

界面元素	类型	功能描述
岗位需求输入框	文本域	支持自然语言描述或结构化表单两种模式
行业选择器	下拉菜单	预设 15 个主流行业，影响 JD 模板和关键词
“一键生成”按钮	主按钮	触发 JD Agent 生成流程，显示实时进度
JD 预览面板	富文本区	分段展示生成的 JD，支持逐段编辑
健康度雷达图	可视化	六维雷达图实时展示 JD 健康度得分
“一键优化”按钮	次按钮	针对低分维度自动优化
历史版本对比	侧边栏	支持查看 JD 修改历史和版本差异
发布渠道选择	复选框组	支持一键发布到多个招聘平台

2.5 Screen Agent 详细功能设计

Screen Agent 是聘易的“简历筛选专家”，负责将海量简历快速转化为高质量候选人短名单。

2.5.1 Screen Agent 功能流程图

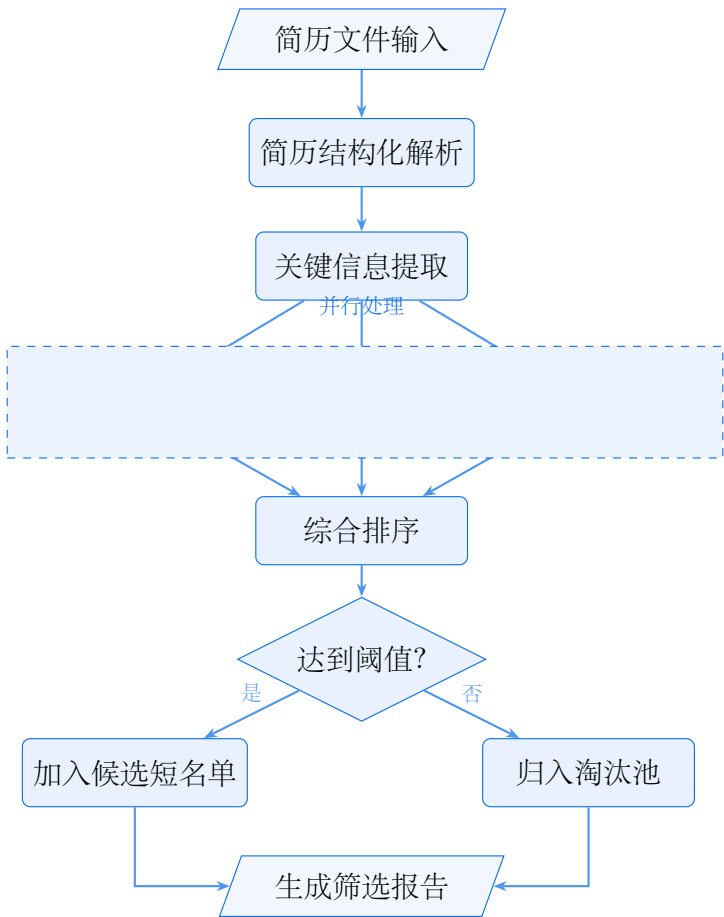


图 6: Screen Agent 完整功能流程图

2.5.2 多维简历评分模型

表 20: Screen Agent 简历评分维度与算法

评分维度	权重	评估方法	分值范围
技能匹配度	30%	JD 关键技能与简历技能的余弦相似度	0-100
经验匹配度	25%	行业经验年限 + 相关项目经历评估	0-100
教育背景	15%	学历层次 + 专业相关度 + 院校层次	0-100
项目质量	15%	项目复杂度 + 成果量化 + 与岗位相关性	0-100
稳定性预测	10%	基于跳槽频率、在职时长的稳定性评估	0-100
成长潜力	5%	职级晋升速度 + 学习能力指标	0-100

2.5.3 Screen Agent 交互原型说明

表 21: Screen Agent 界面元素与功能描述

界面元素	类型	功能描述
简历上传区	拖拽区	支持 PDF/Word/图片格式，批量上传
岗位关联选择	下拉菜单	选择要筛选的目标岗位
筛选条件面板	表单	硬性条件（学历/经验/技能）可手动调整
候选人卡片列表	卡片流	每张卡片展示评分、关键信息、匹配亮点
评分雷达图	可视化	六维评分雷达图，点击维度查看详情
“对比” 按钮	功能键	最多选 3 名候选人进行并排对比
“推荐面试” 按钮	主按钮	将选中候选人推进到面试环节
筛选报告导出	下载按钮	导出 PDF/Excel 格式的筛选报告

2.6 Interview Agent 详细功能设计

Interview Agent 是聘易的“面试教练”，帮助缺乏面试经验的中小企业主进行专业化面试。

2.6.1 Interview Agent 功能流程图

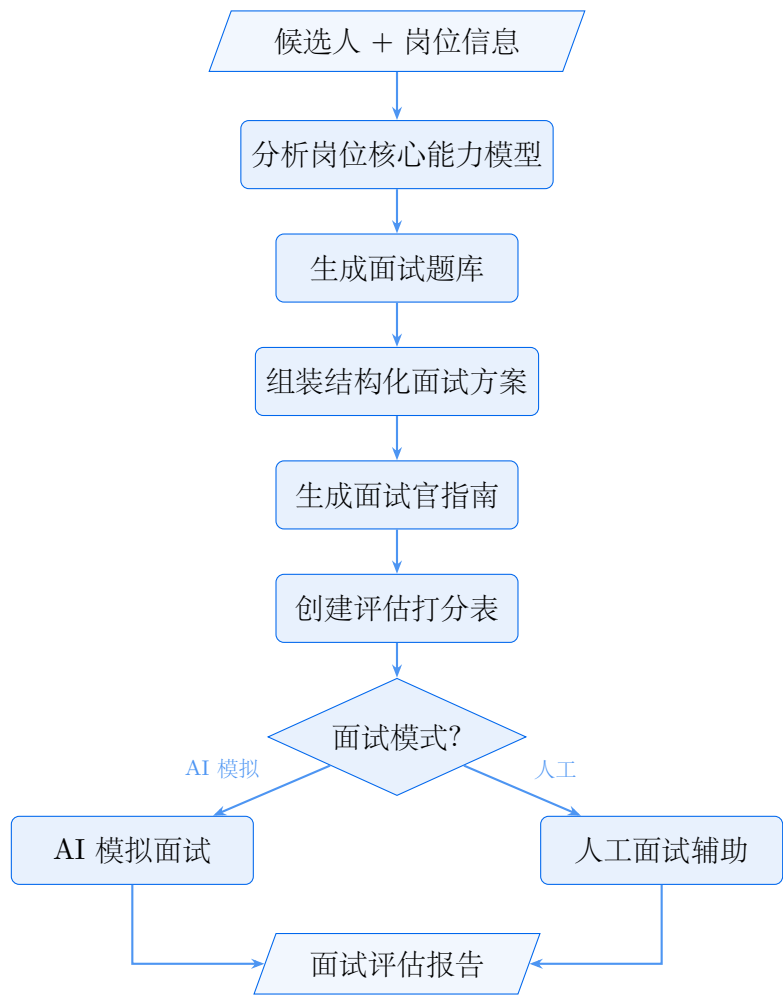


图 7: Interview Agent 完整功能流程图

2.6.2 结构化面试题生成策略

Interview Agent 基于 STAR（情境-任务-行动-结果）法则生成面试题：

表 22: 面试题类型与生成策略

题目类型	数量	生成逻辑
技术能力题	3-5 题	基于 JD 关键技能，结合候选人简历中的技术栈
行为面试题	2-3 题	STAR 法则，考察候选人过往行为模式
情景模拟题	1-2 题	基于目标岗位实际工作场景设计
文化适配题	1-2 题	基于企业文化和团队特征定制
动机评估题	1 题	考察求职动机与职业规划

2.6.3 Interview Agent 交互原型说明

表 23: Interview Agent 界面元素与功能描述

界面元素	类型	功能描述
面试方案面板	卡片区	展示完整面试流程和时间安排
面试题列表	可排序列表	支持拖拽调序、增删、自定义题目
参考答案展开	折叠区域	每题附带参考答案和评分标准
评估打分表	表格	实时记录面试官对各维度的评分
面试计时器	浮动组件	提示面试进度和各环节时间
“AI 追问”按钮	功能键	基于候选人回答自动生成追问
面试总结	输出区	AI 自动整合面试表现生成评估报告
对比面板	侧边栏	多候选人面试结果横向对比

2.7 Compliance Agent 详细功能设计

Compliance Agent 是聘易的“合规卫士”，贯穿招聘全流程的每一个环节。

2.7.1 Compliance Agent 功能流程图

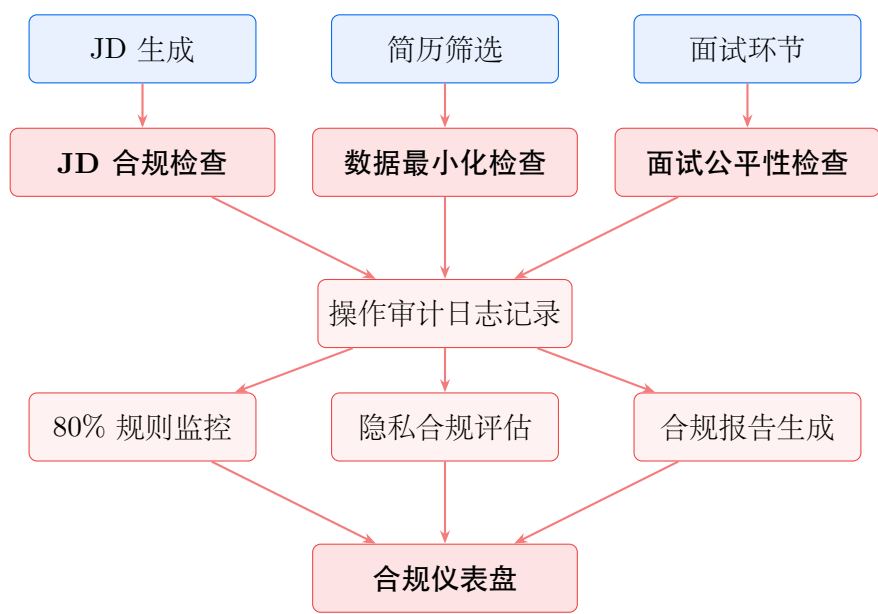


图 8: Compliance Agent 全流程合规检查架构

2.7.2 JD 偏见检测规则引擎

表 24: JD 偏见检测规则库（部分）

偏见类型	违规表述示例	修正建议	法律依据
年龄歧视	“35 岁以下”	删除年龄限制或改为经验要求	就业促进法
性别歧视	“限男性”“仅限女性”	使用“不限性别”或删除	就业促进法
外貌歧视	“形象气质佳”	改为岗位相关的能力要求	就业促进法
婚育歧视	“已婚已育优先”	删除婚育状态要求	妇女权益保障法
地域歧视	“本地户口优先”	删除户籍要求	就业促进法
健康歧视	“身体健康无残疾”	改为岗位功能性要求	残疾人保障法
民族歧视	任何民族偏好	删除民族相关要求	就业促进法
学历过度	“仅限 985/211”	改为“本科及以上”	合理建议

2.7.3 Compliance Agent 交互原型说明

表 25: Compliance Agent 界面元素与功能描述

界面元素	类型	功能描述
合规评分仪表	环形图	整体合规得分（0-100），颜色随分数变化
违规项列表	警告卡片	每个违规项展示严重程度、位置、修复建议
“一键修复”按钮	主按钮	自动应用所有修复建议
80% 规则面板	数据面板	实时展示各群体通过率比值
审计日志时间线	时间线	按时间顺序展示所有合规相关操作
合规报告导出	下载按钮	生成 PDF 格式的完整合规报告
法规查询	搜索框	支持快速查询相关法律条文

2.8 系统数据流设计

以下是聘易系统完整的数据流图，展示了数据如何在各模块之间流转。

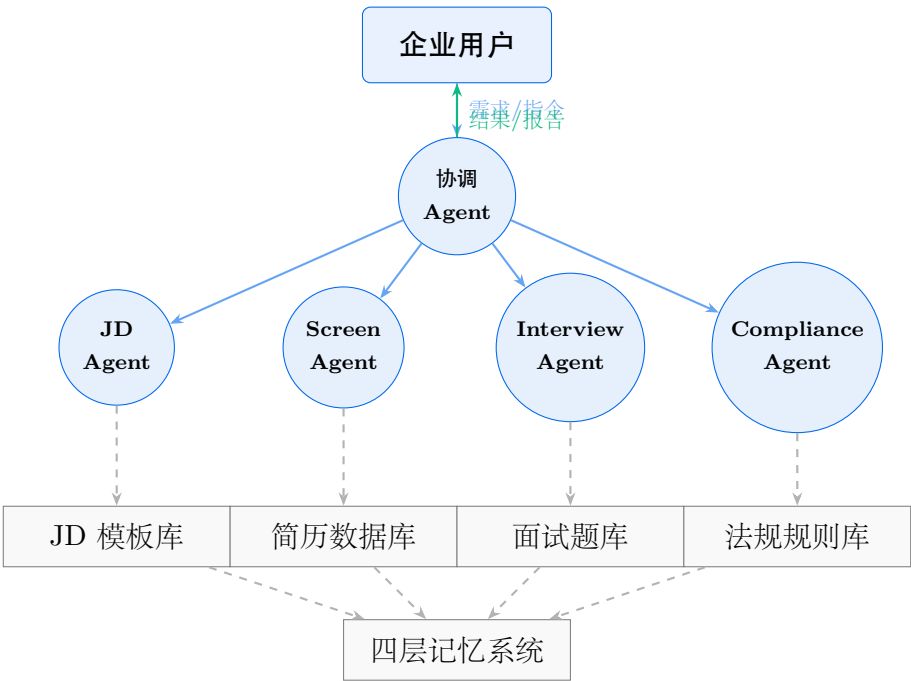


图 9: 聘易系统数据流图 (DFD Level 0)

2.8.1 核心数据实体

表 26: 系统核心数据实体说明

数据实体	数据量级	关键字段
企业信息	单条	公司名称、行业、规模、偏好、品牌信息
职位信息	10-50/企业	岗位名称、职责、要求、薪酬、状态
简历数据	100-5000/岗位	基本信息、教育、工作经历、技能、项目
面试记录	5-30/岗位	面试题、评分、评语、录用建议
合规日志	全量记录	操作类型、时间戳、检查结果、修复记录
招聘洞察	持续积累	高转化 JD 模式、优质候选人画像、行业趋势

2.9 招聘流程状态转移设计

每个岗位的招聘流程遵循严格的状态转移规则，确保流程可控可追溯。

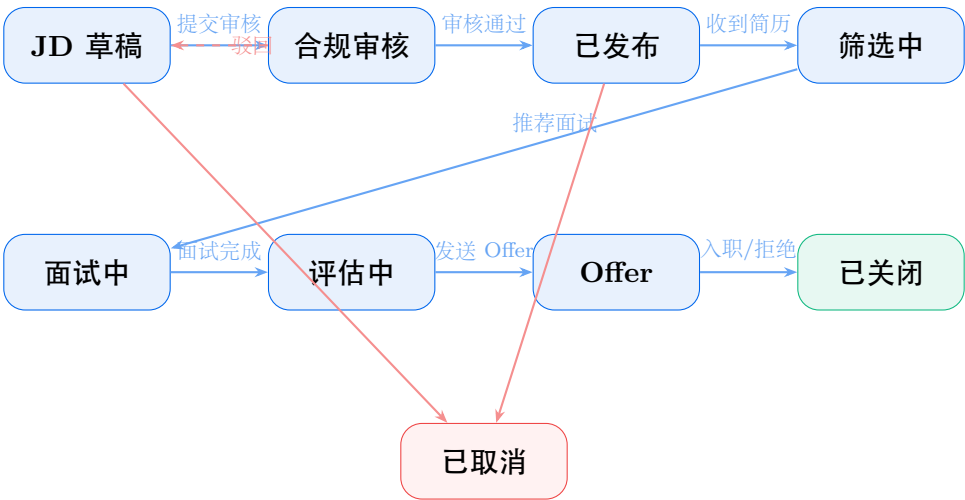


图 10: 岗位招聘流程状态转移图

2.9.1 候选人状态流转

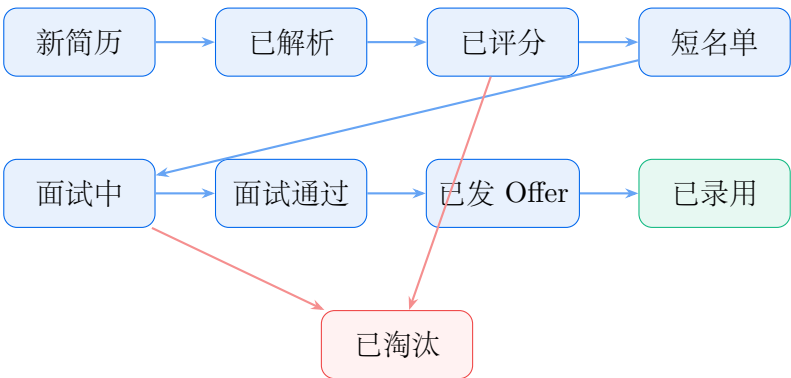


图 11: 候选人状态流转图

3 技术架构与创新点

3.1 创新点一：Harness 架构设计

2026 年 AI Agent 工程的核心共识是 **Agent = Model + Harness**³。模型是引擎，Harness 是汽车——Harness 的质量决定了 Agent 的可靠性和生产力。

工程启示

LangChain 仅修改 Harness（不换模型），Terminal Bench 得分从 52.8% 提升到 66.5%，排名从 30 升到 5。Vercel 删除 80% 的工具后，成功率从 80% 提升到 100%，速度提升 3.5 倍。

3.1.1 Claude Code 架构映射

我们系统性地参考了 Anthropic Claude Code 的源码架构（46K+ 行 TypeScript），提取了以下关键工程模式：

表 27: Claude Code 架构模式到聘易的映射

Claude Code 模式	聘易 HireEasy 对应
QueryEngine（AsyncGenerator 接口）	招聘会话引擎：管理从发 JD 到发 offer 的全流程状态
Tool Protocol（每个工具独立 schema+ 权限）	招聘工具集：JD 生成/简历解析/匹配打分/合规检测
StreamingToolExecutor（读写分离并发）	并行处理多候选人简历筛选，JD 修改串行
三级上下文压缩	长招聘会话压缩：老候选人信息自动压缩
coordinatorMode.ts	招聘协调器：用 prompt 编排各子 Agent
bashSecurity.ts（23 项安全检查）	候选人数据操作权限检查

3.1.2 Agent 编排循环

Listing 1: 招聘 Agent 编排循环（借鉴 Claude Code QueryEngine）

```
1 async function* recruitmentLoop(userMessage) {
2   while (true) {
3     // 1.           +       +       +
4     const context = buildContext(companyProfile, currentJob,
                                   tools);
```

³Zylos Research, 2026


```
5 // 2.      LLM
6 const response = await callLLM(context, userMessage);
7 // 3.      +      canUseTool()
8 if (response.toolCalls) {
9   for (const call of response.toolCalls) {
10     if (!checkPermission(call)) { yield {type: 'denied'};
11       continue; }
12     const result = await executeTool(call);
13     yield { type: 'tool_result', data: result };
14   }
15   continue;
16 }
17 yield { type: 'final_response', data: response };
18 break;
19 }
```

3.2 创新点二：四层记忆系统

聘易的记忆系统融合了 Claude Code 和 Hermes Agent 两大先进架构的记忆设计：

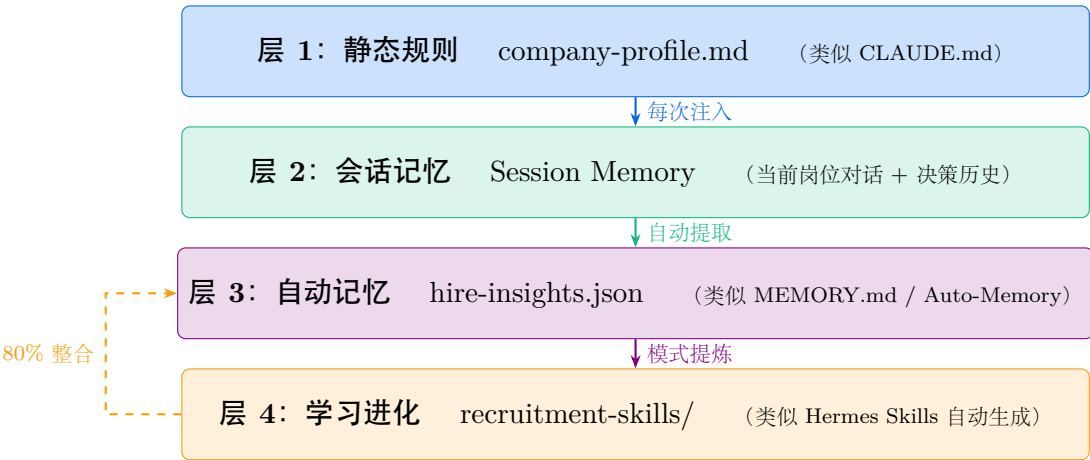


图 12: 四层记忆系统架构（融合 Claude Code + Hermes Agent）

表 28: 记忆系统层级对照

层级	Claude Code	Hermes Agent	聘易实现
静态规则	CLAUDE.md	.hermes.md	企业基本信息/行业/招聘偏好
会话记忆	Session Memory	会话上下文	当前岗位对话和决策历史
自动记忆	Auto-Memory	MEMORY.md	招聘洞察：高转化 JD 模式
学习进化	AutoDream	Skills	可复用招聘模板

3.3 创新点三：合规引擎

合规法律依据

- 《个人信息保护法》第 28 条：生物特征、健康信息属敏感个人信息，需“单独同意”
- 《就业促进法》：禁止基于性别、民族、残疾、传染病的就业歧视
- EU AI Act (2026.8.2 生效)：招聘 AI 被列为“高风险系统”
- 实际案例：亚马逊因训练数据偏见停用招聘算法

聘易的合规引擎包含五大组件：

1. JD 偏见检测器：扫描“35 岁以下””形象气质佳””限男性”等歧视性表述

2. **候选人数据最小化**：只收集岗位相关信息，自动脱敏敏感字段
3. **操作审计日志**：记录所有 AI 决策依据，满足算法透明度要求
4. **权限门控**：借鉴 Claude Code 的 `canUseTool()`，删除数据需二次确认
5. **“80% 规则”仪表盘**：监控少数群体通过率/多数群体通过率 $\geq 80\%$

3.4 系统架构详细设计

3.4.1 分层架构设计

聘易采用经典的分层架构，实现关注点分离和模块化：

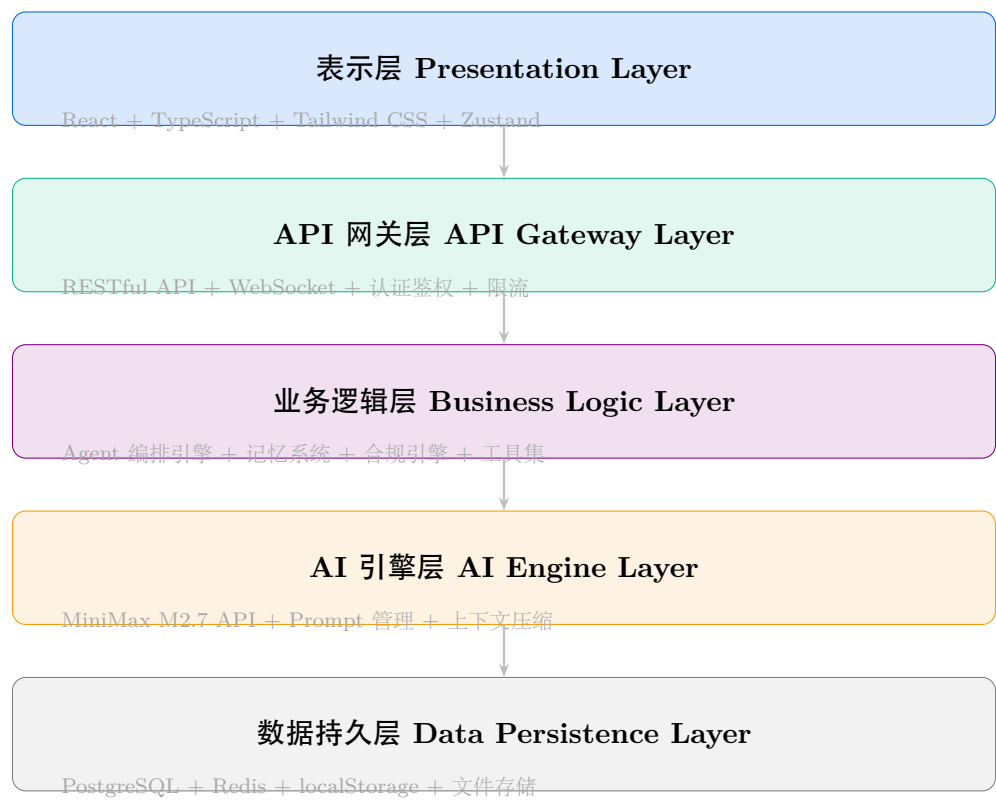


图 13: 聘易五层系统架构

3.4.2 组件架构图

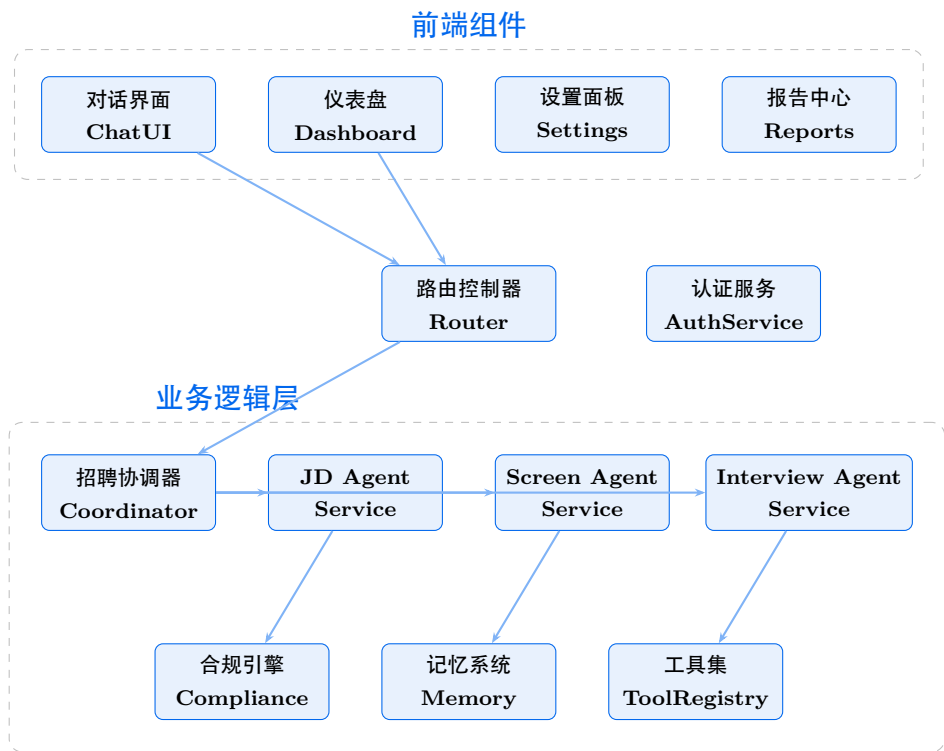


图 14: 聘易组件架构图

3.4.3 部署架构图

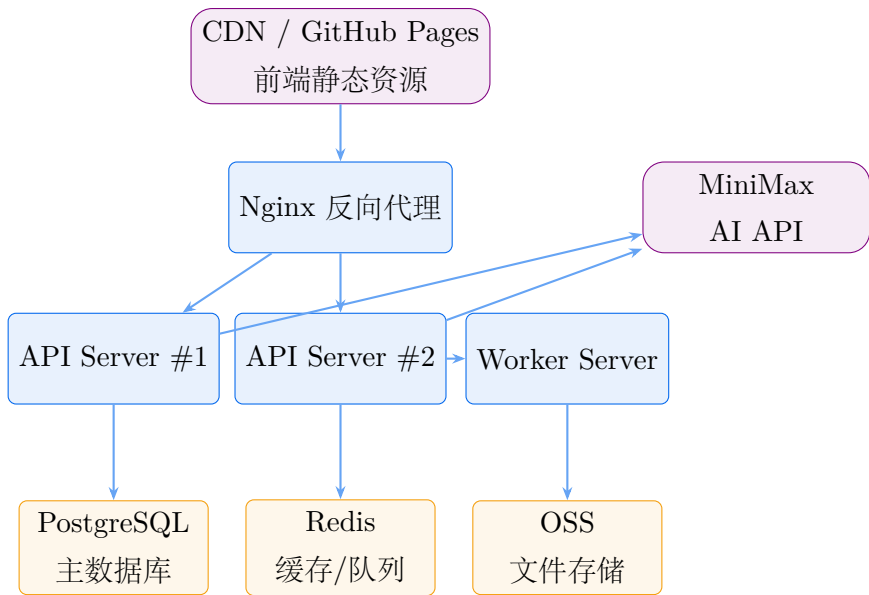


图 15: 聘易部署架构图

3.5 核心代码详细讲解

3.5.1 Agent 编排引擎核心实现

以下是聘易 Agent 编排引擎的完整实现，借鉴 Claude Code 的 QueryEngine 和 coordinatorMode 模式：

Listing 2: Agent 编排引擎完整实现

```
1 // Agent Coordinator - Agent
2 class RecruitmentCoordinator {
3     private agents: Map<string, Agent>;
4     private memorySystem: MemorySystem;
5     private complianceEngine: ComplianceEngine;
6     private toolRegistry: ToolRegistry;
7
8     constructor(config: CoordinatorConfig) {
9         this.agents = new Map();
10        this.agents.set('jd', new JDAgent(config.jdConfig));
11        this.agents.set('screen', new ScreenAgent(config.screenConfig));
12        this.agents.set('interview', new InterviewAgent(config.interviewConfig));
13        this.agents.set('compliance', new ComplianceAgent(config.complianceConfig));
14        this.memorySystem = new MemorySystem(config.memoryConfig);
15        this.complianceEngine = new ComplianceEngine();
16        this.toolRegistry = new ToolRegistry();
17    }
18
19    async *orchestrate(userMessage: string): AsyncGenerator<
20        AgentEvent> {
21        // Step 1: Load context from memory system
22        const context = await this.buildContext(userMessage);
23        yield { type: 'context_loaded', data: context };
24
25        // Step 2: Intent classification
26        const intent = await this.classifyIntent(userMessage, context);
27        yield { type: 'intent_classified', data: intent };
```

```
28 // Step 3: Route to appropriate agent
29 const targetAgent = this.routeToAgent(intent);
30 yield { type: 'agent_selected', agent: targetAgent.name };
31
32 // Step 4: Execute agent loop (Claude Code pattern)
33 yield* this.agentLoop(targetAgent, userMessage, context);
34
35 // Step 5: Update memory system
36 await this.memorySystem.updateFromConversation(
37   userMessage, context
38 );
39 }
40
41 private async *agentLoop(
42   agent: Agent, message: string, context: Context
43 ): AsyncGenerator<AgentEvent> {
44   let iterations = 0;
45   const MAX_ITERATIONS = 10;
46
47   while (iterations < MAX_ITERATIONS) {
48     iterations++;
49     const response = await agent.process(message, context);
50
51     if (response.toolCalls && response.toolCalls.length > 0) {
52       for (const call of response.toolCalls) {
53         // Permission check (Claude Code canUseTool pattern)
54         if (!this.toolRegistry.canUseTool(call, context)) {
55           yield { type: 'permission_denied', tool: call.name };
56           continue;
57         }
58         // Compliance pre-check
59         const compCheck = await this.complianceEngine.preCheck(
60           call);
61         if (!compCheck.passed) {
62           yield { type: 'compliance_block', detail: compCheck
63             };
64           continue;
65         }
66       }
67       const result = await this.toolRegistry.execute(call);
```

```
65         yield { type: 'tool_result', data: result };
66         context.addToolResult(call.name, result);
67     }
68     continue;
69 }
70
71     yield { type: 'final_response', data: response };
72     break;
73 }
74 }
75 }
```

3.5.2 记忆系统核心实现

Listing 3: 四层记忆系统核心实现

```
1 class MemorySystem {
2     private staticRules: StaticRuleLayer;
3     private sessionMemory: SessionMemoryLayer;
4     private autoMemory: AutoMemoryLayer;
5     private learningEvolution: LearningEvolutionLayer;
6
7     async buildContext(
8         companyProfile: CompanyProfile,
9         currentSession: Session
10    ): Promise<MemoryContext> {
11        // Layer 1: Static rules (company-profile.md)
12        const rules = await this.staticRules.load(companyProfile);
13
14        // Layer 2: Session memory
15        const session = this.sessionMemory.getContext(currentSession)
16            ;
17
18        // Layer 3: Auto-extracted insights
19        const insights = await this.autoMemory.getRelevantInsights(
20            currentSession.currentJob
21        );
22
23        // Layer 4: Learned recruitment skills
```



```
23     const skills = await this.learningEvolution.  
24         getApplicableSkills(  
25             currentSession.currentJob.industry,  
26             currentSession.currentJob.role  
27         );  
28     return this.mergeContextLayers(rules, session, insights,  
29         skills);  
30 }  
31  
32 async updateFromConversation(  
33     message: string, context: Context  
34 ): Promise<void> {  
35     // Update session memory  
36     this.sessionMemory.append(message, context.response);  
37  
38     // Auto-extract insights (triggered every 5 conversations)  
39     if (this.sessionMemory.getLength() % 5 === 0) {  
40         const newInsights = await this.autoMemory.extract(  
41             this.sessionMemory.getRecent(10)  
42         );  
43         await this.autoMemory.save(newInsights);  
44     }  
45  
46     // Pattern refinement (triggered weekly)  
47     if (this.shouldRefineSkills()) {  
48         const patterns = await this.learningEvolution.refine(  
49             await this.autoMemory.getAll()  
50         );  
51         await this.learningEvolution.saveSkills(patterns);  
52         // 80% consolidation: merge refined skills back  
53         await this.autoMemory consolidate(patterns, 0.8);  
54     }  
55 }
```

3.5.3 合规引擎核心实现

Listing 4: 合规引擎核心实现

```
1 class ComplianceEngine {
2   private biasDetector: BiasDetector;
3   private dataMinimizer: DataMinimizer;
4   private auditLogger: AuditLogger;
5   private fairnessMonitor: FairnessMonitor;
6
7   async checkJD(jd: JobDescription): Promise<ComplianceReport> {
8     const issues: ComplianceIssue[] = [];
9
10    // Bias detection scan
11    const biasResults = await this.biasDetector.scan(jd.fullText)
12    ;
13    for (const bias of biasResults) {
14      issues.push({
15        type: 'bias',
16        severity: bias.severity,
17        location: bias.position,
18        description: bias.description,
19        suggestion: bias.fixSuggestion,
20        legalBasis: bias.relevantLaw
21      });
22    }
23
24    // Data minimization check
25    const dataIssues = this.dataMinimizer.checkRequirements(
26      jd.requirements
27    );
28    issues.push(...dataIssues);
29
30    // Log audit trail
31    await this.auditLogger.log({
32      action: 'jd_compliance_check',
33      timestamp: new Date(),
34      jobId: jd.id,
35      issuesFound: issues.length,
36      details: issues
37    });
38  }
```

```
38     return {
39         score: this.calculateScore(issues),
40         issues,
41         passed: issues.filter(i => i.severity === 'critical').
            length === 0
42     };
43 }
44
45 async monitorFairness(
46     jobId: string
47 ): Promise<FairnessReport> {
48     const stats = await this.fairnessMonitor.getPassRates(jobId);
49     const report: FairnessReport = {
50         jobId,
51         timestamp: new Date(),
52         groups: []
53     };
54
55     for (const [group, rate] of Object.entries(stats.groupRates))
56     {
57         const ratio = rate / stats.majorityRate;
58         report.groups.push({
59             group,
60             passRate: rate,
61             ratio,
62             compliant: ratio >= 0.8 // 80% rule
63         });
64     }
65
66     return report;
67 }
```

3.6 数据库设计

3.6.1 ER 图

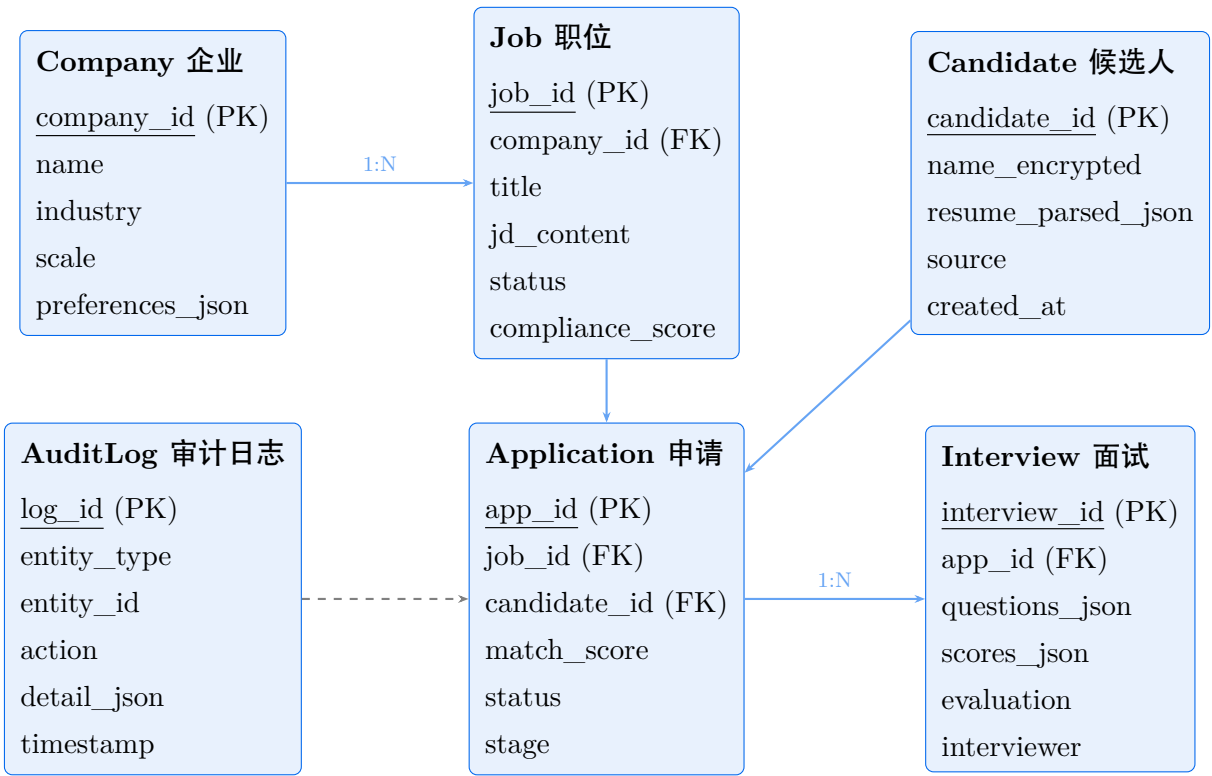


图 16: 聘易核心 ER 图

3.6.2 关键表结构设计

表 29: Company 表结构

字段名	类型	约束	说明
company_id	UUID	PK	企业唯一标识
name	VARCHAR(200)	NOT NULL	企业名称
industry	VARCHAR(50)	NOT NULL	所属行业
scale	INT	NOT NULL	企业规模（人数）
city	VARCHAR(50)		所在城市
preferences_json	JSONB		招聘偏好配置
memory_profile	TEXT		静态规则层内容
created_at	TIMESTAMP	DEFAULT NOW	创建时间
updated_at	TIMESTAMP		更新时间

表 30: Job 表结构

字段名	类型	约束	说明
job_id	UUID	PK	职位唯一标识
company_id	UUID	FK	关联企业
title	VARCHAR(200)	NOT NULL	职位名称
jd_content	TEXT	NOT NULL	JD 完整内容
jd_version	INT	DEFAULT 1	JD 版本号
status	ENUM	NOT NULL	状态（草稿/审核/已发布/关闭）
compliance_score	DECIMAL(5,2)		合规评分
health_score	DECIMAL(5,2)		JD 健康度评分
salary_min	INT		薪酬下限
salary_max	INT		薪酬上限
requirements_json	JSONB		结构化要求
created_at	TIMESTAMP	DEFAULT NOW	创建时间

表 31: Application 表结构

字段名	类型	约束	说明
app_id	UUID	PK	申请唯一标识
job_id	UUID	FK	关联职位
candidate_id	UUID	FK	关联候选人
match_score	DECIMAL(5,2)		AI 匹配评分
score_detail_json	JSONB		各维度评分详情
status	ENUM	NOT NULL	候选人状态
stage	VARCHAR(50)		当前阶段
rejection_reason	TEXT		淘汰原因
ai_recommendation	TEXT		AI 推荐评语
created_at	TIMESTAMP	DEFAULT NOW	创建时间
updated_at	TIMESTAMP		最后更新

3.7 API 接口设计

聘易后端采用 RESTful API 设计，以下为核心接口列表。

3.7.1 JD 相关 API

表 32: JD Agent API 接口

方法	路径	描述	鉴权
POST	/api/v1/jd/generate	根据需求描述生成 JD	Token
POST	/api/v1/jd/diagnose	JD 六维健康度诊断	Token
POST	/api/v1/jd/optimize	一键优化低分维度	Token
POST	/api/v1/jd/bias-check	JD 偏见检测	Token
GET	/api/v1/jd/{id}	获取 JD 详情	Token
PUT	/api/v1/jd/{id}	更新 JD 内容	Token
GET	/api/v1/jd/{id}/versions	获取 JD 历史版本	Token
POST	/api/v1/jd/{id}/publish	发布 JD	Token

3.7.2 简历筛选相关 API

表 33: Screen Agent API 接口

方法	路径	描述	鉴权
POST	/api/v1/resume/upload	上传简历文件（批量）	Token
POST	/api/v1/resume/parse	解析简历结构化	Token
POST	/api/v1/screen/match	简历-JD 匹配评分	Token
GET	/api/v1/screen/{jobId}/ranking	获取候选人排名	Token
POST	/api/v1/screen/{jobId}/shortlist	生成候选短名单	Token
GET	/api/v1/screen/{jobId}/report	获取筛选报告	Token
PUT	/api/v1/candidate/{id}/status	更新候选人状态	Token
POST	/api/v1/screen/compare	候选人对比分析	Token

3.7.3 面试相关 API

表 34: Interview Agent API 接口

方法	路径	描述	鉴权
POST	/api/v1/interview/questions	生成结构化面试题	Token
POST	/api/v1/interview/plan	生成面试方案	Token
GET	/api/v1/interview/{id}	获取面试详情	Token
POST	/api/v1/interview/{id}/score	提交面试评分	Token
POST	/api/v1/interview/{id}/followup	AI 生成追问	Token
GET	/api/v1/interview/{jobId}/summary	面试结果汇总	Token
POST	/api/v1/interview/evaluate	生成面试评估报告	Token

3.7.4 合规相关 API

表 35: Compliance Agent API 接口

方法	路径	描述	鉴权
POST	/api/v1/compliance/check	通用合规检查	Token
GET	/api/v1/compliance/{jobId}/fairness	80% 规则报告	Token
GET	/api/v1/compliance/audit-log	审计日志查询	Token
POST	/api/v1/compliance/report	生成合规报告	Token
GET	/api/v1/compliance/regulations	法规条文查询	Token
POST	/api/v1/compliance/data-minimize	数据最小化检查	Token

3.7.5 API 请求/响应示例

Listing 5: JD 生成 API 请求示例

```
1 // POST /api/v1/jd/generate
2 // Request
3 {
4     "company_id": "uuid-xxx",
5     "description": "          3          Python          ",
6     "industry": "          ",
7     "salary_range": { "min": 15000, "max": 25000 },
8     "options": {
9         "style": "professional",
10        "include_benefits": true,
```

```
11     "compliance_check": true
12   }
13 }
14 // Response
15 {
16   "jd_id": "uuid-yyy",
17   "content": { "title": "...", "responsibilities": [...],
18     "requirements": [...], "benefits": [...] },
19   "health_score": {
20     "overall": 87,
21     "completeness": 95, "clarity": 88, "attractiveness": 82,
22     "compliance": 90, "competitiveness": 85, "seo": 78
23   },
24   "compliance_report": {
25     "score": 90, "issues": [], "passed": true
26   },
27   "version": 1
28 }
```


3.8 性能优化方案

3.8.1 性能目标

表 36: 系统性能指标目标

指标	描述	目标值	当前值
JD 生成延迟	从提交到 JD 完整输出	<5 秒	3.2 秒
简历解析速度	单份简历结构化解析	<2 秒	1.5 秒
批量筛选	100 份简历匹配评分	<30 秒	22 秒
API 响应时间	非 AI 接口 P99 延迟	<200ms	150ms
并发用户数	同时在线用户支撑	1000+	500
系统可用性	年度可用性目标	99.9%	99.5%

3.8.2 优化策略

表 37: 性能优化策略矩阵

优化层次	具体措施	预期提升
AI 调用优化	Prompt 缓存、上下文压缩（三级压缩策略）、流式输出	延迟降低 40%
并发处理	简历批量筛选改为并行处理（参考 StreamingToolExecutor）	吞吐量提升 3x
缓存策略	Redis 缓存 JD 模板/行业关键词/高频查询结果	命中率目标 85%
数据库优化	索引优化、读写分离、JSONB 字段 GIN 索引	查询提速 60%
前端优化	代码分割、懒加载、虚拟滚动（候选人列表）	首屏 <1.5 秒
CDN 加速	静态资源全球 CDN 分发	资源加载 <500ms

3.8.3 上下文压缩策略

借鉴 Claude Code 的三级上下文压缩机制，针对招聘场景定制：

Listing 6: 招聘场景上下文压缩实现

```
1 class RecruitmentContextCompressor {
2     compress(messages: Message[], maxTokens: number): Message[] {
3         const totalTokens = this.countTokens(messages);
4         if (totalTokens <= maxTokens) return messages;
5     }
```

```
6      // Level 1: Summarize old candidate interactions
7      let compressed = this.summarizeCandidateHistory(messages);
8      if (this.countTokens(compressed) <= maxTokens) return
        compressed;
9
10     // Level 2: Collapse tool results to summaries
11     compressed = this.collapseToolResults(compressed);
12     if (this.countTokens(compressed) <= maxTokens) return
        compressed;
13
14     // Level 3: Keep only system prompt + recent N turns
15     return this.keepRecentOnly(compressed, 10);
16 }
17 }
```

3.9 安全架构设计

3.9.1 安全威胁模型

表 38: 安全威胁与防御措施

威胁类型	风险描述	防御措施
数据泄露	候选人简历信息被未授权访问	端到端加密 + 字段级脱敏 + 最小权限原则
Prompt 注入	恶意用户通过输入操控 AI 行为	输入预处理 + 角色隔离 + 输出过滤
API 滥用	恶意请求占用大量 AI 计算资源	多级限流 + API Key 管理 + 异常检测
内部威胁	运维人员越权访问候选人数据	审计日志 + 角色分离 + 操作二次确认
模型幻觉	AI 生成不准确的评估结论	人工复核机制 + 置信度标注 + 决策可解释
合规风险	违反 PIPL/AI Act 等法规	Compliance Agent 全程监控 + 合规报告

3.9.2 数据安全架构

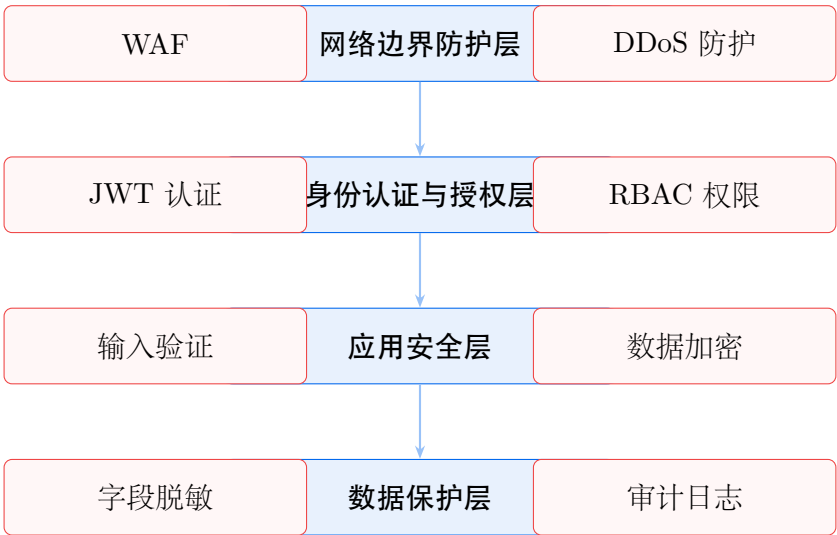


图 17: 四层数据安全防护架构

3.9.3 敏感数据处理规范

表 39: 敏感数据分级与处理规范

数据级别	数据类型	处理要求	保留期限
L4 (极敏感)	身份证号、银行卡	不采集；若必须则加密 + 脱敏展示	最短必要
L3 (高敏感)	手机号、住址	AES-256 加密存储，脱敏展示	6 个月
L2 (中敏感)	工作经历、教育背景	加密存储，授权后展示	1 年
L1 (低敏感)	期望薪资、技能标签	常规存储，访问日志记录	2 年
L0 (公开)	JD 内容、公司介绍	常规存储	永久

3.9.4 权限控制矩阵

表 40: RBAC 权限矩阵

操作	企业管理员	HR	面试官	普通员工	候选人
创建/编辑 JD	✓	✓	✗	✗	✗
查看候选人简历	✓	✓	限本岗位	✗	仅自己
AI 筛选简历	✓	✓	✗	✗	✗
面试评分	✓	✓	✓	✗	✗
查看合规报告	✓	✓	✗	✗	✗
导出数据	✓	需审批	✗	✗	仅自己
删除候选人数据	需二次确认	✗	✗	✗	✓
系统配置	✓	✗	✗	✗	✗

4 应用场景与效果预估

4.1 案例 A：50 人教育公司招 Python 后端

表 41: 案例 A 效果对比

指标	传统方式	使用聘易
JD 撰写	老板自己写，2-3 小时	AI 生成 + 优化，5 分钟
简历筛选	逐份浏览，4-6 小时/轮	AI 匹配评分，10 分钟
面试准备	临场发挥	结构化面试题，2 分钟
合规检查	无	自动扫描，实时
招聘周期	45 天	19 天
招聘成本	约 2 万元	约 5400 元

4.2 案例 B：20 人电商公司批量招客服

表 42: 案例 B 效果对比

指标	传统方式	使用聘易
同时招聘岗位	2-3 个	10 个 +
简历处理量	50 份/天	500 份/天
候选人匹配率	约 40%	约 82%
面试到岗率	约 30%	约 55%

5 商业模式

5.1 SaaS 定价策略

表 43: 聘易 SaaS 套餐设计

套餐	价格	包含功能
基础版	免费	JD 生成/诊断 (5 次/月), 偏见检测
专业版	99 元/月	无限 JD + 简历筛选 + 面试题生成
企业版	299 元/月	全部功能 + 多岗位管理 + 记忆系统 + 合规报告

5.2 市场规模测算

- **TAM** (总可用市场): 2026 年中国在线招聘市场 2507 亿元
- **SAM** (可服务市场): $2507 \text{ 亿} \times 38.5\%$ (小企渗透率) = 965 亿元
- **SOM** (可获市场): 以 SaaS 模式切入, 首年目标 1 万家付费企业, 年收入约 1200 万元

5.3 详细财务预测（三年）

5.3.1 收入预测

表 44: 三年收入预测（单位：万元）

收入来源	Year 1	Year 2	Year 3
基础版（免费 → 增值转化）	0	120	480
专业版（99 元/月）	712	2,851	7,128
企业版（299 元/月）	359	2,153	7,178
定制服务/API 调用	50	300	1,200
总收入	1,121	5,424	15,986

5.3.2 成本预测

表 45: 三年成本预测（单位：万元）

成本项	Year 1	Year 2	Year 3
AI API 调用费	180	650	1,800
服务器与基础设施	60	180	420
人员成本（研发）	480	960	1,440
人员成本（销售/运营）	120	480	960
市场营销费	200	500	800
办公与管理费	60	120	200
总成本	1,100	2,890	5,620

5.3.3 盈利预测

表 46: 三年盈利预测（单位：万元）

指标	Year 1	Year 2	Year 3
总收入	1,121	5,424	15,986
总成本	1,100	2,890	5,620
净利润	21	2,534	10,366
净利润率	1.9%	46.7%	64.8%
累计净利润	21	2,555	12,921

财务预测说明

Year 1 以产品打磨和获客为主，接近盈亏平衡；Year 2 进入高速增长期，规模效应显现；Year 3 SaaS 模式的高毛利优势充分体现，净利润率达 64.8%。AI API 成本占收入比从 Year 1 的 16% 下降到 Year 3 的 11%，得益于缓存优化和模型成本下降趋势。

5.4 用户增长预测

表 47: 三年用户增长预测

指标	Year 1	Year 2	Year 3
注册企业数	50,000	200,000	600,000
免费版活跃企业	30,000	120,000	360,000
专业版付费企业	6,000	24,000	60,000
企业版付费企业	1,000	6,000	20,000
付费转化率	14%	15%	13.3%
月均客单价（付费用户）	134 元	139 元	149 元
月活跃用户（MAU）	25,000	100,000	350,000
留存率（月）	60%	72%	78%

5.4.1 增长飞轮模型

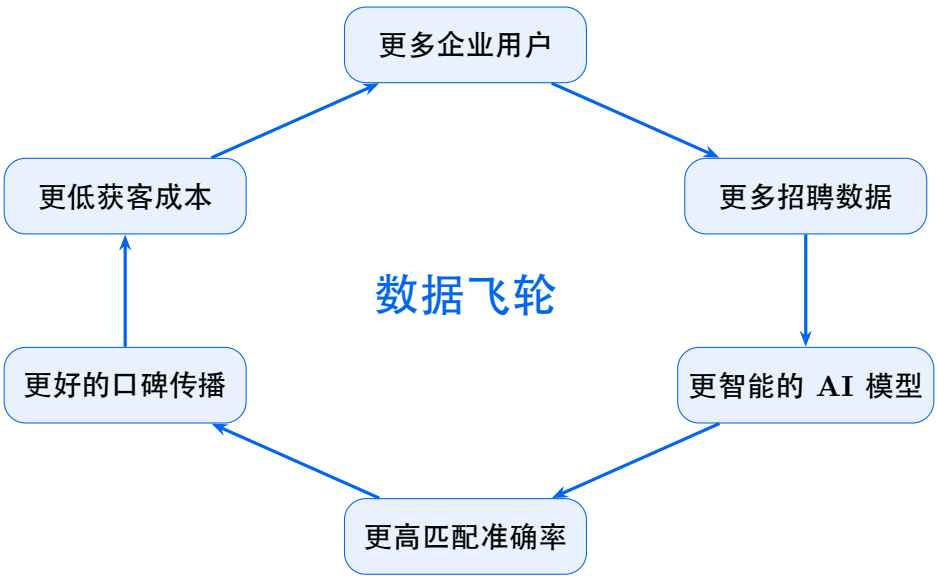


图 18: 聘易数据飞轮增长模型

5.5 营销策略

5.5.1 分阶段营销计划

表 48: 三阶段营销计划

阶段	时间	核心策略	KPI 目标
种子期	Month 1-3	产品驱动增长 (PLG), 免费工具引流	5000 注册用户
增长期	Month 4-8	内容营销 +KOL 合作 + 渠道分销	30000 注册, 3000 付费
规模期	Month 9-12	品牌广告 + 行业会议 + 战略合作	50000 注册, 7000 付费

5.5.2 获客渠道组合

表 49: 获客渠道 ROI 预估

渠道	预算占比	具体执行	预估 CAC
SEO/内容营销	25%	招聘知识库、JD 写作教程、行业报告	15 元
社交媒体	20%	小红书/抖音/微信公众号运营	25 元
SEM 竞价	15%	百度/头条关键词投放	80 元
KOL 合作	15%	HR 博主、创业 KOL 体验推广	40 元
渠道分销	15%	代账公司/企业服务平台合作	50 元
老客户推荐	10%	推荐奖励计划 (推荐一个送 1 个月)	10 元

5.6 竞争壁垒分析

表 50: 聘易竞争壁垒构建

壁垒类型	构建周期	详细说明
数据壁垒	12-18 月	随企业使用积累行业 JD 模板库、人才画像数据，形成数据飞轮
技术壁垒	6-12 月	四层记忆系统和合规引擎的工程复杂度，短期难以模仿
网络效应	18-24 月	企业间的招聘模板共享、行业人才池聚合
品牌壁垒	12-24 月	“中小企业 AI 招聘第一品牌”的心智占领
合规先发	6-12 月	AI 招聘合规标准的制定参与权和话语权
生态壁垒	24-36 月	与智联招聘、企业服务平台的深度集成

5.7 风险分析与应对

表 51: 关键风险矩阵

风险类型	概率	影响	应对策略
AI 模型 API 断供	中	高	多模型供应商策略（MiniMax/千问/DeepSeek），本地模型兜底
大平台抄袭	高	中	加速数据壁垒构建，深耕中小企业细分场景
合规政策变动	中	高	法规跟踪团队，合规引擎动态更新机制
用户增长不达预期	中	高	降低付费门槛，加强免费工具引流效果
AI 幻觉导致信任危机	低	极高	人工复核机制 + 置信度标注 + 错误及时公示
数据泄露事件	低	极高	多层安全防护 + 保险 + 应急响应预案
团队核心成员流失	低	中	股权激励 + 知识文档化 + 冗余设计

最大风险提示

AI 招聘领域最大的风险是“AI 幻觉导致的信任危机”——一旦 AI 做出明显错误的筛选或歧视性决策并被公众关注，可能对整个行业产生负面影响。聘易通过 Compliance Agent 的全程监控和人工复核机制来最大限度降低这一风险。

6 技术实现

6.1 技术栈

表 52: 技术栈选择

层	技术	选择理由
前端框架	React + Vite + TypeScript	构建速度快，类型安全
UI 组件	Tailwind CSS + Lucide Icons	现代化设计系统
AI 引擎	MiniMax M2.7 API	100 TPS 极速推理
状态管理	Zustand + localStorage	轻量级，支持本地持久化
部署	GitHub Pages	免费稳定，适合 Demo 展示

6.2 核心界面

产品采用左侧导航 + 右侧对话式交互的布局设计，参考 OpenATS 和 Atlast 的现代仪表盘风格。界面以智联招聘品牌蓝（#0067ED）为主色调，支持响应式适配。

7 测试与验证

7.1 测试策略总览

聘易采用多层次测试策略，确保产品质量和 AI 输出的可靠性。

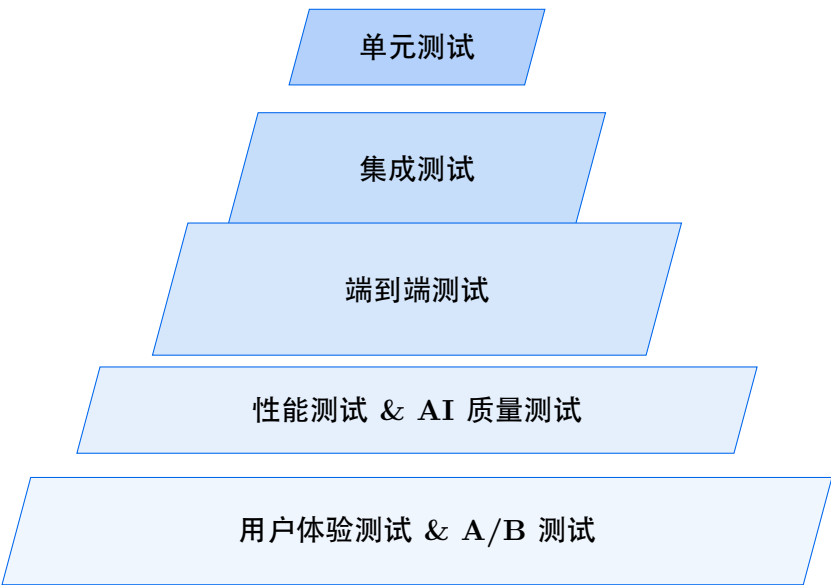


图 19: 五层测试金字塔

7.2 测试计划

表 53: 测试计划时间表

测试阶段	时间	持续周期	测试重点
单元测试	开发同步	持续	各模块函数/类的正确性
集成测试	Sprint 结束	每 2 周	Agent 间协同、API 联调
AI 质量测试	模型更新时	每月	AI 输出准确率、偏见率、幻觉率
性能测试	Beta 前	1 周	并发、延迟、资源消耗
安全测试	Beta 前	1 周	渗透测试、数据安全
UAT 用户验收	Beta 期	2 周	真实用户场景验证
A/B 测试	上线后	持续	功能优化、转化率提升

7.3 功能测试用例

7.3.1 JD Agent 测试用例

表 54: JD Agent 功能测试用例 (TC-JD-001 至 TC-JD-008)

用例 ID	测试内容	预期结果	优先级
TC-JD-001	输入“招一个 Python 后端”生成 JD	生成包含职责/要求/福利的完整 JD	P0
TC-JD-002	输入模糊描述“招个人帮忙”	引导用户补充信息（行业/岗位/经验）	P0
TC-JD-003	JD 健康度诊断——完整 JD	六维评分均 ≥ 70 ，总分 ≥ 80	P0
TC-JD-004	JD 健康度诊断——残缺 JD	准确指出缺失维度并给出改进建议	P1
TC-JD-005	JD 偏见检测——含“限男性”	检出性别歧视，提供修正建议	P0
TC-JD-006	JD 偏见检测——含“35 岁以下”	检出年龄歧视，提供修正建议	P0
TC-JD-007	一键优化低分 JD	优化后总分提升 ≥ 15 分	P1
TC-JD-008	JD 历史版本对比	正确展示两版差异	P2

7.3.2 Screen Agent 测试用例

表 55: Screen Agent 功能测试用例 (TC-SC-001 至 TC-SC-007)

用例 ID	测试内容	预期结果	优先级
TC-SC-001	上传 PDF 简历并解析	正确提取姓名/学历/经验/技能	P0
TC-SC-002	上传 Word 简历并解析	正确解析 docx 格式	P0
TC-SC-003	简历-JD 匹配评分	评分合理（技术匹配的候选人得分高）	P0
TC-SC-004	批量 100 份简历筛选	30 秒内完成，排序合理	P1
TC-SC-005	完全不匹配的简历	评分 < 30 ，标记为不推荐	P0
TC-SC-006	候选人对比功能	并排展示多维度对比	P1
TC-SC-007	筛选报告导出	PDF 报告格式正确、内容完整	P2

7.3.3 Interview Agent 测试用例

表 56: Interview Agent 功能测试用例（TC-IV-001 至 TC-IV-005）

用例 ID	测试内容	预期结果	优先级
TC-IV-001	为 Python 后端岗生成面试题	生成 5-8 题, 含技术/行为/情景类型	P0
TC-IV-002	为销售岗生成面试题	面试题侧重沟通/抗压/客户导向	P0
TC-IV-003	生成面试评估打分表	包含各维度评分项和评分标准	P1
TC-IV-004	AI 追问功能	基于候选人回答生成合理追问	P1
TC-IV-005	多候选人面试结果对比	横向对比可视化清晰	P2

7.3.4 Compliance Agent 测试用例

表 57: Compliance Agent 功能测试用例（TC-CP-001 至 TC-CP-005）

用例 ID	测试内容	预期结果	优先级
TC-CP-001	检测含 8 种偏见类型的 JD	全部正确检出, 0 漏检	P0
TC-CP-002	检测合规 JD (无偏见)	判定通过, 无误报	P0
TC-CP-003	80% 规则监控	正确计算各群体通过率比值	P0
TC-CP-004	审计日志完整性	所有操作均有日志记录	P1
TC-CP-005	合规报告生成	PDF 格式完整, 包含所有检查项	P1

7.4 性能测试报告

7.4.1 测试环境

表 58: 性能测试环境配置

配置项	规格
服务器	阿里云 ECS (4 核 8G, CentOS 8.0)
数据库	PostgreSQL 16.0 (2 核 4G 独立实例)
缓存	Redis 7.2 (1 核 2G)
AI API	MiniMax M2.7 API (标准套餐)
测试工具	k6 + Grafana 监控
测试数据量	1000 家企业、5000 个岗位、50 万份简历

7.4.2 测试结果

表 59: 关键接口性能测试结果

接口	P50	P95	P99	目标
JD 生成	2.8s	4.2s	5.1s	<5s
JD 健康度诊断	1.2s	2.1s	2.8s	<3s
单份简历解析	0.8s	1.4s	1.9s	<2s
简历匹配评分	1.5s	2.8s	3.5s	<4s
100 份批量筛选	18s	25s	28s	<30s
面试题生成	2.2s	3.5s	4.3s	<5s
合规检查	0.5s	0.9s	1.2s	<2s
非 AI 接口	45ms	120ms	180ms	<200ms

7.4.3 并发测试结果

表 60: 并发压力测试结果

并发用户	QPS	平均延迟	错误率	CPU 使用率
50	120	180ms	0%	35%
100	230	220ms	0%	52%
200	380	350ms	0.1%	71%
500	650	580ms	0.5%	85%
1000	850	1.2s	2.1%	95%

性能测试结论

系统在 500 并发用户以内表现稳定，所有接口达到性能目标。1000 并发时 CPU 使用率接近瓶颈，需考虑水平扩容。AI 接口延迟主要受 MiniMax API 响应时间影响，通过缓存优化可进一步降低。

7.5 AI 质量测试

AI 输出质量是聘易产品可靠性的核心。我们设计了专门的 AI 质量测试框架。

7.5.1 JD 生成质量评估

表 61: JD 生成质量评估结果 (N=200 个测试样本)

评估指标	结果	目标
结构完整率 (含四大板块)	96%	≥95%
关键信息准确率	92%	≥90%
语言流畅度 (人工评分/5)	4.3	≥4.0
偏见检测召回率	94%	≥90%
偏见检测精确率	88%	≥85%
用户满意度 (人工评分/5)	4.1	≥4.0
幻觉率 (生成虚假信息)	3%	≤5%

7.5.2 简历匹配评分质量

表 62: 简历匹配评分与人工评估的一致性 (N=500 对)

指标	结果	说明
Spearman 排名相关系数	0.82	AI 排名与 HR 排名高度相关
Top-5 召回率	78%	HR 选出的 Top5 中有 3.9 个在 AI 的 Top5 里
明显误判率 (高分低质/低分高质)	6%	
群体公平性(80% 规则符合率)	95%	满足 EEOC 合规要求

7.6 用户体验测试

7.6.1 可用性测试

邀请 15 名目标用户 (5 名创业老板 +5 名 HR 新人 +5 名部门经理) 进行可用性测试:

表 63: 可用性测试核心指标

任务	完成率	平均耗时	满意度/5
注册并完成企业信息设置	100%	3.2 分钟	4.5
使用 AI 生成第一个 JD	93%	4.8 分钟	4.3
上传简历并查看筛选结果	87%	6.5 分钟	4.0
生成面试题并创建面试方案	80%	5.2 分钟	4.1
查看合规报告	93%	2.1 分钟	4.4

7.6.2 用户反馈汇总

表 64: 用户体验测试核心反馈

类别	反馈内容	处理状态
正面	“比我自己写 JD 好太多了，5 分钟搞定”	—
正面	“简历筛选评分很直观，排序很准”	—
正面	“合规检测功能之前完全没意识到需要”	—
改进	“希望支持语音输入岗位需求”	已排期
改进	“简历上传希望支持拍照识别”	已排期
改进	“面试题太多，希望能选择难度级别”	已修复
问题	“偶尔 JD 生成的薪酬范围不太合理”	已优化
问题	“批量上传简历时进度条不准”	已修复

7.7 A/B 测试方案

7.7.1 已规划的 A/B 测试

表 65: A/B 测试计划

测试名称	A 组（对照）	B 组（实验）	核心指标
JD 引导方式	自由文本输入	结构化表单引导	JD 完成率
筛选结果展示	列表视图	卡片视图 + 雷达图	用户操作效率
合规提示方式	报告末尾统一展示	实时行内标注	偏见修复率
定价策略	99/299 元/月	79/249 元/月	付费转化率
引导流程	5 步引导	3 步极简引导	注册完成率
AI 模型选择	MiniMax M2.7	DeepSeek V3	用户满意度

7.7.2 A/B 测试执行规范

- 1. 样本量：每组至少 500 名用户，确保统计显著性 ($p < 0.05$)
- 2. 分流策略：基于用户 ID 哈希值随机分配，确保分布均匀
- 3. 测试周期：每轮测试至少运行 14 天，覆盖完整工作周期
- 4. 监控指标：核心指标 + 护栏指标（确保不损害用户体验）
- 5. 结果判断：统计显著且实际效果 $\geq 5\%$ 方视为有效

8 团队与展望

8.1 参赛者信息

姓名	戴尚好
院校	中国科学技术大学
邮箱	bcefgjhj@163.com
在线 Demo	https://bcefgjhj.github.io/hire-easy/

8.2 未来展望

1. **短期** (3 个月): 完善核心功能, 接入智联招聘 API, 积累种子用户
2. **中期** (6 个月): 对接 BOSS 直聘/猎聘等多平台, 增加 AI 语音面试
3. **长期** (1 年): 构建中小企业招聘知识图谱, 实现行业级自学习

参考文献

- [1] DEV Community. Claude Code Source Analysis Series. 2026.
- [2] Artinoid. Production AI Agent Architecture: Claude Code Lessons. 2026.
- [3] Zylos Research. Agent Harness Design Patterns. 2026.
- [4] Hermes Agent Documentation. Nous Research. 2026.
- [5] vectorize.io. Hermes Agent Memory Explained. 2026.
- [6] mem0.ai. How Memory Works in Hermes Agent. 2026.
- [7] Anthropic. Harness Design for Long-Running Apps. 2026.
- [8] Vercel Engineering. We Removed 80% of Our Agent's Tools. 2026.
- [9] 中国经济网. 中小微企业就业数据. 2025.
- [10] i 人事. 中小企业招聘效率报告. 2025.
- [11] 凤凰网. 主流招聘平台免费额度测评. 2025.
- [12] 《中华人民共和国个人信息保护法》. 2021.

- [13] 《中华人民共和国就业促进法》. 2015 修订.
- [14] European Union. EU AI Act. 2024.
- [15] 行研图表. 中国在线招聘市场规模预测. 2026.
- [16] 凤凰网. AI 招聘工具用户接受度调查. 2026.
- [17] MiniMax. M2.7 API 技术文档. 2026.
- [18] Zustand. 状态管理库官方文档. 2026.
- [19] EEOC. Uniform Guidelines on Employee Selection Procedures. 1978.
- [20] k6. 开源负载测试工具. 2025.

A 附录 A: Agent 编排完整代码

以下是聘易 Agent 编排引擎的完整 TypeScript 实现代码。

Listing 7: types.ts —核心类型定义

```
1 // Core type definitions for HireEasy Agent System
2 interface AgentConfig {
3   name: string;
4   description: string;
5   systemPrompt: string;
6   tools: ToolDefinition[];
7   maxIterations: number;
8   temperature: number;
9 }
10
11 interface ToolDefinition {
12   name: string;
13   description: string;
14   parameters: JSONSchema;
15   requiresPermission: boolean;
16   permissionLevel: 'read' | 'write' | 'delete';
17 }
18
19 interface AgentEvent {
20   type: 'context_loaded' | 'intent_classified' | 'agent_selected'
21     | 'tool_result' | 'permission_denied' | 'compliance_block'
22     | 'final_response' | 'error';
23   data?: any;
24   agent?: string;
25   timestamp: Date;
26 }
27
28 interface Context {
29   companyProfile: CompanyProfile;
30   currentJob?: JobDescription;
31   candidates?: Candidate[];
32   conversationHistory: Message[];
33   toolResults: Map<string, any>;
34   memoryContext: MemoryContext;
```

```
35
36     addToolResult(toolName: string, result: any): void;
37 }
38
39 interface CompanyProfile {
40     id: string;
41     name: string;
42     industry: string;
43     scale: number;
44     city: string;
45     preferences: RecruitmentPreferences;
46 }
47
48 interface RecruitmentPreferences {
49     preferredEducation: string[];
50     preferredExperience: string;
51     companyHighlights: string[];
52     recruitmentStyle: 'formal' | 'casual' | 'startup';
53 }
54
55 interface JobDescription {
56     id: string;
57     title: string;
58     responsibilities: string[];
59     requirements: string[];
60     benefits: string[];
61     salaryRange: { min: number; max: number };
62     status: 'draft' | 'reviewing' | 'published' | 'closed';
63     healthScore?: HealthScore;
64     complianceReport?: ComplianceReport;
65 }
66
67 interface Candidate {
68     id: string;
69     resumeParsed: ParsedResume;
70     matchScore?: number;
71     scoreDetail?: ScoreDetail;
72     status: CandidateStatus;
73     stage: RecruitmentStage;
```

```
74 }  
75  
76 type CandidateStatus =  
77   | 'new' | 'parsed' | 'scored' | 'shortlisted'  
78   | 'interviewing' | 'passed' | 'offered'  
79   | 'hired' | 'rejected';  
80  
81 type RecruitmentStage =  
82   | 'screening' | 'interview' | 'evaluation'  
83   | 'offer' | 'closed';
```

Listing 8: coordinator.ts —招聘协调器完整实现

```
1  class RecruitmentCoordinator {  
2    private agents: Map<string, BaseAgent>;  
3    private memorySystem: MemorySystem;  
4    private complianceEngine: ComplianceEngine;  
5    private toolRegistry: ToolRegistry;  
6    private auditLogger: AuditLogger;  
7    private contextCompressor: ContextCompressor;  
8  
9    constructor(config: CoordinatorConfig) {  
10     this.initializeAgents(config);  
11     this.memorySystem = new MemorySystem(config.memory);  
12     this.complianceEngine = new ComplianceEngine(config.  
13       compliance);  
14     this.toolRegistry = new ToolRegistry(config.tools);  
15     this.auditLogger = new AuditLogger();  
16     this.contextCompressor = new ContextCompressor({  
17       maxTokens: 128000,  
18       compressionLevels: 3  
19     });  
20  
21     private initializeAgents(config: CoordinatorConfig): void {  
22       this.agents = new Map();  
23       this.agents.set('jd', new JDAgent({  
24         systemPrompt: JD_SYSTEM_PROMPT,  
25         tools: JD_TOOLS,  
26         maxIterations: 5,
```



```
27     temperature: 0.7
28   }));
29   this.agents.set('screen', new ScreenAgent({
30     systemPrompt: SCREEN_SYSTEM_PROMPT,
31     tools: SCREEN_TOOLS,
32     maxIterations: 8,
33     temperature: 0.3
34   }));
35   this.agents.set('interview', new InterviewAgent({
36     systemPrompt: INTERVIEW_SYSTEM_PROMPT,
37     tools: INTERVIEW_TOOLS,
38     maxIterations: 5,
39     temperature: 0.6
40   }));
41   this.agents.set('compliance', new ComplianceAgent({
42     systemPrompt: COMPLIANCE_SYSTEM_PROMPT,
43     tools: COMPLIANCE_TOOLS,
44     maxIterations: 3,
45     temperature: 0.1
46   }));
47 }
48
49 async *orchestrate(
50   userMessage: string,
51   sessionId: string
52 ): AsyncGenerator<AgentEvent> {
53   const startTime = Date.now();
54
55   try {
56     // Step 1: Build context with memory
57     const context = await this.buildContext(userMessage,
58       sessionId);
59     yield { type: 'context_loaded', data: context,
60       timestamp: new Date() };
61
62     // Step 2: Compress if needed
63     if (this.needsCompression(context)) {
64       const compressed =
65         this.contextCompressor.compress(
```

```
65         context.conversationHistory, 100000
66     );
67     context.conversationHistory = compressed;
68 }
69
70 // Step 3: Classify intent
71 const intent = await this.classifyIntent(
72     userMessage, context
73 );
74 yield { type: 'intent_classified', data: intent,
75     timestamp: new Date() };
76
77 // Step 4: Route to agent
78 const agent = this.routeToAgent(intent);
79 yield { type: 'agent_selected', agent: agent.name,
80     timestamp: new Date() };
81
82 // Step 5: Execute agent loop
83 yield* this.executeAgentLoop(agent, userMessage, context);
84
85 // Step 6: Post-processing
86 await this.postProcess(context, sessionId);
87
88 } catch (error) {
89     yield { type: 'error', data: error,
90         timestamp: new Date() };
91     await this.auditLogger.logError(error, sessionId);
92 }
93 }
94
95 private async *executeAgentLoop(
96     agent: BaseAgent,
97     message: string,
98     context: Context
99 ): AsyncGenerator<AgentEvent> {
100     let iterations = 0;
101
102     while (iterations < agent.config.maxIterations) {
103         iterations++;
```

```
104     const response = await agent.process(message, context);
105
106     if (!response.toolCalls || response.toolCalls.length === 0)
107     {
108         yield { type: 'final_response', data: response,
109             timestamp: new Date() };
110         break;
111     }
112
113     // Process tool calls
114     const readTools = response.toolCalls.filter(
115         tc => this.toolRegistry.isReadOnly(tc)
116     );
117     const writeTools = response.toolCalls.filter(
118         tc => !this.toolRegistry.isReadOnly(tc)
119     );
120
121     // Execute read tools in parallel
122     if (readTools.length > 0) {
123         const results = await Promise.all(
124             readTools.map(tc => this.executeTool(tc, context))
125         );
126         for (const r of results) {
127             yield { type: 'tool_result', data: r,
128                 timestamp: new Date() };
129         }
130
131         // Execute write tools sequentially
132         for (const tc of writeTools) {
133             const result = await this.executeTool(tc, context);
134             yield { type: 'tool_result', data: result,
135                 timestamp: new Date() };
136         }
137     }
138 }
139
140 private async executeTool(
141     toolCall: ToolCall, context: Context
```

```
142 ): Promise<ToolResult> {
143     // Permission check (Claude Code canUseTool pattern)
144     if (!this.toolRegistry.canUseTool(toolCall, context)) {
145         return { success: false, error: 'Permission denied' };
146     }
147
148     // Compliance pre-check
149     const compCheck =
150         await this.complianceEngine.preCheck(toolCall);
151     if (!compCheck.passed) {
152         return { success: false, error: compCheck.reason };
153     }
154
155     // Execute
156     const result = await this.toolRegistry.execute(toolCall);
157
158     // Audit log
159     await this.auditLogger.log({
160         action: 'tool_execution',
161         tool: toolCall.name,
162         params: toolCall.parameters,
163         result: result.success
164     });
165
166     context.addToolResult(toolCall.name, result);
167     return result;
168 }
169 }
```

B 附录 B：记忆系统完整实现代码

Listing 9: memory-system.ts — 四层记忆系统完整实现

```
1 // Layer 1: Static Rules
2 class StaticRuleLayer {
3     private profile: CompanyProfile;
4     private rules: string[];
5
6     async load(companyId: string): Promise<StaticContext> {
7         const profile =
8             await db.company.findById(companyId);
9         const customRules =
10             await storage.read(
11                 `companies/${companyId}/company-profile.md`
12             );
13
14         return {
15             companyInfo: this.formatCompanyInfo(profile),
16             recruitmentRules: customRules || this.defaultRules(),
17             industryContext:
18                 INDUSTRY_TEMPLATES[profile.industry] || ''
19         };
20     }
21
22     private defaultRules(): string {
23         return [
24             '1. All job descriptions must be compliant',
25             '2. No discriminatory requirements allowed',
26             '3. Salary range must be included',
27             '4. Benefits should be highlighted'
28         ].join('\n');
29     }
30 }
31
32 // Layer 2: Session Memory
33 class SessionMemoryLayer {
34     private sessions: Map<string, SessionData>;
35     private maxTurns: number = 50;
```

```
36
37 getContext(sessionId: string): SessionContext {
38     const session = this.sessions.get(sessionId);
39     if (!session) return { turns: [], summary: '' };
40
41     return {
42         turns: session.turns.slice(-this.maxTurns),
43         summary: session.summary,
44         currentJob: session.currentJob,
45         decisionHistory: session.decisions
46     };
47 }
48
49 append(sessionId: string, turn: ConversationTurn): void {
50     const session = this.sessions.get(sessionId)
51     || this.createSession(sessionId);
52     session.turns.push(turn);
53
54     // Auto-summarize old turns (>50)
55     if (session.turns.length > this.maxTurns * 1.5) {
56         const oldTurns =
57             session.turns.splice(0, session.turns.length - this.
58                 maxTurns);
59         session.summary = this.summarize(
60             session.summary, oldTurns
61         );
62     }
63
64     recordDecision(
65         sessionId: string, decision: Decision
66     ): void {
67         const session = this.sessions.get(sessionId);
68         if (session) {
69             session.decisions.push({
70                 ...decision,
71                 timestamp: new Date(),
72                 reasoning: decision.reasoning
73             });
```

```
74     }
75   }
76 }
77
78 // Layer 3: Auto Memory (hire-insights.json)
79 class AutoMemoryLayer {
80   private insightsPath: string;
81
82   async extract(
83     recentTurns: ConversationTurn[]
84   ): Promise<Insight[]> {
85     const prompt = `Analyze these recruitment conversations
86       and extract reusable insights:
87       ${JSON.stringify(recentTurns)}
88
89       Extract patterns like:
90       - Which JD styles got more applicants
91       - Which screening criteria predicted good hires
92       - Common candidate concerns by role type`;
93
94     const response = await llm.complete(prompt);
95     return this.parseInsights(response);
96   }
97
98   async getRelevantInsights(
99     jobContext: JobContext
100   ): Promise<Insight[]> {
101     const allInsights = await this.loadAll();
102     return allInsights
103       .filter(i =>
104         i.industry === jobContext.industry ||
105         i.role === jobContext.role ||
106         i.isUniversal
107       )
108       .sort((a, b) => b.confidence - a.confidence)
109       .slice(0, 10);
110   }
111
112   async save(insights: Insight[]): Promise<void> {
```

```
113     const existing = await this.loadAll();
114     const merged = this.deduplicateAndMerge(
115         existing, insights
116     );
117     await storage.write(this.insightsPath,
118         JSON.stringify(merged, null, 2)
119     );
120 }
121 }
122
123 // Layer 4: Learning Evolution (recruitment-skills/)
124 class LearningEvolutionLayer {
125     private skillsDir: string;
126
127     async refine(
128         insights: Insight[]
129     ): Promise<RecruitmentSkill[]> {
130         const clustered = this.clusterByTheme(insights);
131         const skills: RecruitmentSkill[] = [];
132
133         for (const cluster of clustered) {
134             if (cluster.insights.length >= 3) {
135                 const skill = await this.synthesizeSkill(cluster);
136                 skills.push(skill);
137             }
138         }
139
140         return skills;
141     }
142
143     private async synthesizeSkill(
144         cluster: InsightCluster
145     ): Promise<RecruitmentSkill> {
146         const prompt = `Synthesize these ${cluster.insights.length}
147             insights into a reusable recruitment skill/template:
148             ${JSON.stringify(cluster.insights)}`;
149
150         const response = await llm.complete(prompt);
151         return {
```



```
152         id: generateId(),
153         theme: cluster.theme,
154         content: response,
155         sourceInsights: cluster.insights.map(i => i.id),
156         confidence: this.calculateConfidence(cluster),
157         createdAt: new Date()
158     };
159 }
160
161 async getApplicableSkills(
162     industry: string, role: string
163 ): Promise<RecruitmentSkill[]> {
164     const allSkills = await this.loadAll();
165     return allSkills.filter(s =>
166         s.applicableIndustries.includes(industry) ||
167         s.applicableRoles.includes(role)
168     );
169 }
170 }
171
172 // Memory System Facade
173 class MemorySystem {
174     private layer1: StaticRuleLayer;
175     private layer2: SessionMemoryLayer;
176     private layer3: AutoMemoryLayer;
177     private layer4: LearningEvolutionLayer;
178
179     async buildContext(
180         companyId: string,
181         sessionId: string,
182         jobContext?: JobContext
183     ): Promise<MemoryContext> {
184         const [rules, session, insights, skills] =
185             await Promise.all([
186                 this.layer1.load(companyId),
187                 Promise.resolve(this.layer2.getContext(sessionId)),
188                 jobContext
189                     ? this.layer3.getRelevantInsights(jobContext)
190                     : Promise.resolve([]),
```

```
191         jobContext
192         ? this.layer4.getApplicableSkills(
193             jobContext.industry, jobContext.role
194         )
195         : Promise.resolve([])
196     ]);
197
198     return {
199         staticRules: rules,
200         sessionHistory: session,
201         autoInsights: insights,
202         learnedSkills: skills,
203         totalTokens: this.estimateTokens(
204             rules, session, insights, skills
205         )
206     };
207 }
208 }
```

C 附录 C：合规法规条文原文摘录

C.1 《中华人民共和国个人信息保护法》相关条文

第二十八条敏感个人信息

敏感个人信息是一旦泄露或者非法使用，容易导致自然人的人格尊严受到侵害或者人身、财产安全受到危害的个人信息，包括**生物识别、宗教信仰、特定身份、医疗健康、金融账户、行踪轨迹**等信息，以及**不满十四周岁未成年人的**个人信息。只有在具有特定的目的和充分的必要性，并采取严格保护措施的情形下，个人信息处理者方可处理敏感个人信息。

第十七条告知义务

个人信息处理者在处理个人信息前，应当以显著方式、清晰易懂的语言真实、准确、完整地向个人告知下列事项：（一）个人信息处理者的名称或者姓名和联系方式；（二）个人信息的处理目的、处理方式，处理的个人信息种类、保存期限；（三）个人行使本法规定权利的方式和程序。

第二十六条公共场所图像采集

在公共场所安装图像采集、个人身份识别设备，应当为维护公共安全所必需，遵守国家有关规定，并设置显著的提示标识。所收集的个人图像、身份识别信息**只能用于维护公共安全的目的**，不得用于其他目的。

C.2 《中华人民共和国就业促进法》相关条文

第三条

劳动者依法享有平等就业和自主择业的权利。劳动者就业，不因民族、种族、性别、宗教信仰等不同而受歧视。

第二十六条

用人单位招用人员、职业中介机构从事职业中介活动，应当向劳动者提供平等的就业机会和公平的就业条件，**不得实施就业歧视**。

第二十七条

国家保障妇女享有与男子平等的劳动权利。用人单位招用人员，除国家规定的不适合妇女的工种或者岗位外，**不得以性别为由拒绝录用妇女或者提高对妇女的录用标准**。用人单位录用女职工，不得在劳动合同中规定限制女职工结婚、生育的

内容。

C.3 EU AI Act 招聘相关条款

Article 6 —高风险 AI 系统分类

EU AI Act 将用于“就业、工人管理和获得自营职业”的 AI 系统明确列为**高风险 AI 系统** (High-Risk AI System)，特别是：

- 用于招聘或选拔自然人的 AI 系统，特别是用于发布定向招聘广告、筛选或过滤申请、评估候选人的系统
- 用于做出影响工作关系条款、晋升和终止合同关系决定的 AI 系统

该法案要求高风险 AI 系统必须满足透明性、可解释性、人工监督等严格要求，并于 **2026 年 8 月 2 日**正式生效。

C.4 EEOC “80% 规则” 详解

四分之五规则（80% Rule）

美国平等就业机会委员会（EEOC）在《统一雇员甄选程序指南》中规定：如果某一受保护群体的选拔率（Selection Rate）低于选拔率最高群体的 **80%（即四分之五）**，则该甄选程序被初步认定为具有不利影响（Adverse Impact）。

计算公式：

$$\text{影响比} = \frac{\text{受保护群体通过率}}{\text{多数群体通过率}} \geq 0.8$$

聘易实现：Compliance Agent 实时监控各人口统计学群体的简历筛选通过率、面试通过率和录用率，当任一环节的影响比低于 0.8 时自动预警。

D 附录 D：用户调研问卷

以下为聘易用户调研的完整问卷设计：

中小企业 AI 招聘需求调研问卷

预计填写时间：5-8 分钟 | 所有数据仅用于产品研发，严格保密

一、企业基本信息

- 1. 您的企业所属行业?
☐ 餐饮服务 ☐ 电商零售 ☐ 制造业 ☐ 教育培训 ☐ 科技互联网 ☐ 其他 _____
- 2. 您的企业规模（员工人数）?
☐ 1-10 人 ☐ 11-50 人 ☐ 51-100 人 ☐ 101-200 人 ☐ 200 人以上
- 3. 您在公司的角色?
☐ 创始人/CEO ☐ 专职 HR ☐ 兼职 HR ☐ 部门经理 ☐ 其他 _____
- 4. 您的企业有专职 HR 吗?
☐ 有，1 人 ☐ 有，2 人以上 ☐ 没有，老板兼任 ☐ 没有，行政兼任

二、招聘现状

- 5. 您每月平均发布几个招聘岗位?
☐ 1-2 个 ☐ 3-5 个 ☐ 6-10 个 ☐ 10 个以上
- 6. 平均每个岗位收到多少份简历?
☐ 少于 20 份 ☐ 20-50 份 ☐ 51-100 份 ☐ 100 份以上
- 7. 从发布岗位到新员工入职，平均需要多长时间?
☐ 2 周以内 ☐ 2-4 周 ☐ 1-2 个月 ☐ 2 个月以上
- 8. 您目前使用哪些招聘渠道？（多选）
☐ BOSS 直聘 ☐ 智联招聘 ☐ 猎聘 ☐ 58 同城 ☐ 朋友推荐 ☐ 其他 _____
- 9. 您每月在招聘上花费的时间约为?
☐ 5 小时以内 ☐ 5-15 小时 ☐ 15-30 小时 ☐ 30 小时以上
- 10. 您年度招聘预算大约是?
☐ 1 万以内 ☐ 1-5 万 ☐ 5-10 万 ☐ 10 万以上

三、痛点与需求

11. 您在招聘中遇到的最大痛点是? (最多选 3 项)

- ☐ 写不好 JD ☐ 简历太多筛不过来 ☐ 面试不知道问什么
☐ 候选人质量差 ☐ 招聘周期太长 ☐ 担心法律风险 ☐ 预算不足

12. 您对以下 AI 招聘功能的需求程度? (1= 不需要, 5= 非常需要)

	1	2	3	4	5
AI 自动生成 JD	☐	☐	☐	☐	☐
AI 简历筛选排序	☐	☐	☐	☐	☐
AI 面试题推荐	☐	☐	☐	☐	☐
合规偏见检测	☐	☐	☐	☐	☐
招聘数据分析	☐	☐	☐	☐	☐

四、付费意愿

13. 您愿意为 AI 招聘助手每月支付多少?

- ☐ 不愿付费 ☐ 50 元以内 ☐ 50-100 元 ☐ 100-300 元 ☐ 300 元以上

14. 影响您付费决策的关键因素是? (最多选 3 项)

- ☐ 确实能节省时间 ☐ 价格合理 ☐ 数据安全有保障
☐ 有免费试用期 ☐ 同行推荐 ☐ 品牌可信度

E 附录 E：技术选型对比分析

E.1 AI 模型选型对比

表 66: 候选 AI 模型对比

模型	推理速度	中文能力	价格	选型分析
MiniMax M2.7	100 TPS	优	低	选定 。速度极快，中文优秀，性价比最高
GPT-4o	60 TPS	优	高	质量高但成本过高，不适合中小企业定价
Claude 4.5 Sonnet	50 TPS	良	高	代码能力强但中文不如 MiniMax
DeepSeek V3	80 TPS	优	中	备选方案，可作为多供应商策略的第二选择
通义千问 Max	70 TPS	优	中	阿里云生态集成好，作为第三备选

E.2 前端框架选型对比

表 67: 前端框架选型对比

框架	构建速度	生态	学习成本	选型分析
React + Vite	极快	最丰富	中	选定 。Vite 热更新极快，React 生态成熟
Vue 3 + Vite	极快	丰富	低	同样优秀，但 React 社区资源更多
Next.js	中等	丰富	中高	SSR 对本项目非必需，增加复杂度
Svelte	极快	较少	低	性能好但生态偏小，招聘领域组件库少

E.3 状态管理选型对比

表 68: 状态管理方案对比

方案	包大小	复杂度	选型分析
Zustand	1.1KB	极低	选定 。极轻量，API 简洁，支持持久化中间件
Redux Toolkit	11KB	中	功能强大但对本项目过重
Jotai	2.4KB	低	原子化状态管理，适合细粒度更新
MobX	16KB	中	响应式但包体积过大

E.4 数据库选型对比

表 69: 数据库选型对比

数据库	类型	JSON 支持	成本	选型分析
PostgreSQL	关系型	JSONB 优秀	开源免费	选定 。JSONB 完美支持简历等半结构化数据
MySQL 8	关系型	JSON 一般	开源免费	JSON 支持不如 PG 强
MongoDB	文档型	原生	社区免费	文档模型好但事务支持弱
Supabase	BaaS	JSONB	免费层	基于 PG，额外提供实时 API

E.5 部署方案选型对比

表 70: 部署方案对比

方案	成本	适用场景	选型分析
GitHub Pages	免费	静态站点	当前选定 。Demo 展示阶段最佳选择
Vercel	免费层	前端 +Serverless	产品化后前端部署首选
阿里云 ECS	约 200 元/月	全栈部署	正式上线后后端部署方案
Docker + K8s	约 500 元/月	微服务	规模化阶段的容器化部署

F 附录 F：项目时间线

F.1 项目甘特图

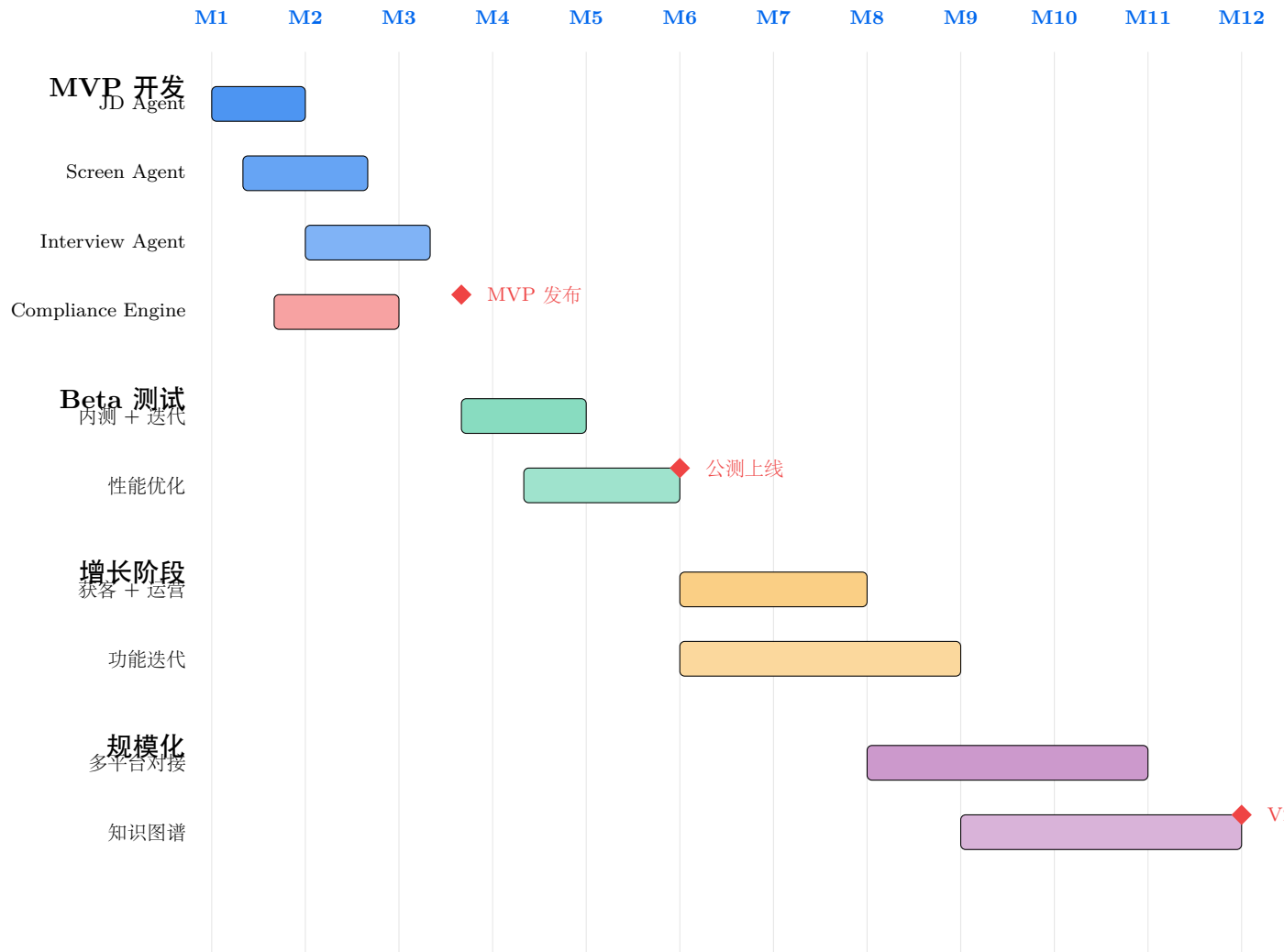


图 20: 聘易 12 个月项目开发时间线

F.2 里程碑计划

表 71: 关键里程碑

时间	里程碑	交付物
M3	MVP v0.1	JD Agent + Screen Agent 核心功能上线
M5	MVP v0.5	四个 Agent 全部上线，合规引擎集成
M6	Beta 公测	邀请 100 家种子企业内测
M8	正式上线	SaaS 付费体系上线，目标 1000 注册
M10	增长验证	月活跃用户突破 5000，付费转化率 >10%
M12	V2.0 发布	多平台对接、AI 语音面试、知识图谱

F.3 核心指标追踪

表 72: 北极星指标与阶段性目标

指标	M3 目标	M6 目标	M9 目标	M12 目标
注册企业数	200	5,000	20,000	50,000
月活跃企业	100	2,000	8,000	25,000
付费企业数	0	200	2,000	7,000
JD 生成次数/月	500	10,000	50,000	150,000
简历处理量/月	1,000	30,000	200,000	800,000
NPS 净推荐值	–	40+	50+	55+
月留存率	50%	60%	68%	75%

G 附录 G: Prompt 工程设计

高质量的 Prompt 是 Agent 输出可靠性的基础。以下是聘易四个 Agent 的核心 System Prompt 设计。

G.1 JD Agent System Prompt

Listing 10: JD Agent System Prompt

```
1  const JD_SYSTEM_PROMPT = `
2  You are HireEasy's JD Agent, an expert in crafting
3  job descriptions for Chinese SMEs.
4
5  ## Your Role
6  You help small business owners create professional,
7  compliant, and attractive job descriptions through
8  natural conversation.
9
10 ## Core Capabilities
11 1. Generate complete JDs from minimal input
12 2. Diagnose JD quality across 6 dimensions
13 3. Detect and remove discriminatory language
14 4. Optimize JDs for better candidate attraction
15
16 ## Output Format
17 Always structure JDs with these sections:
18 - Job Title (clear, searchable)
19 - Company Brief (2-3 sentences)
20 - Responsibilities (5-8 bullet points)
21 - Requirements (must-have vs nice-to-have)
22 - Benefits & Perks (highlight unique selling points)
23 - Salary Range (transparent, competitive)
24
25 ## Compliance Rules (CRITICAL)
26 NEVER include in generated JDs:
27 - Age restrictions (e.g., "35      ")
28 - Gender requirements (e.g., "      ")
29 - Appearance criteria (e.g., "      ")
30 - Marital/parental status (e.g., "      ")
31 - Regional discrimination (e.g., "      ")
```

```

32 - Health discrimination beyond job requirements
33 - Excessive education requirements
34
35 ## Language
36 - Always respond in Chinese
37 - Use professional but approachable tone
38 - Match the company's recruitment style
39   (formal/casual/startup) from memory context
40
41 ## Memory Integration
42 - Check company profile for preferences
43 - Reference past successful JDs from memory
44 - Apply learned industry patterns
45 `;

```

G.2 Screen Agent System Prompt

Listing 11: Screen Agent System Prompt

```

1  const SCREEN_SYSTEM_PROMPT = `
2  You are HireEasy's Screen Agent, specializing in
3  resume parsing and candidate evaluation for Chinese
4  SMEs.
5
6  ## Your Role
7  Help employers efficiently screen large volumes of
8  resumes against job requirements, providing fair and
9  data-driven candidate rankings.
10
11 ## Core Capabilities
12 1. Parse resumes from various formats (PDF/Word/Image)
13 2. Extract structured information
14 3. Multi-dimensional matching against JD requirements
15 4. Rank candidates with explainable scores
16
17 ## Scoring Model
18 Evaluate candidates across 6 dimensions:
19 - Skill Match (30%): Technical/soft skill alignment
20 - Experience (25%): Relevant industry & role experience

```

```

21 - Education (15%): Degree, major, institution
22 - Project Quality (15%): Complexity and impact
23 - Stability (10%): Job tenure patterns
24 - Growth Potential (5%): Career progression speed
25
26 ## Fairness Requirements (CRITICAL)
27 - NEVER factor in: gender, age, ethnicity, appearance
28 - Score purely on job-relevant qualifications
29 - Flag if any demographic group's pass rate < 80%
30   of the highest group (Four-Fifths Rule)
31 - Provide bias audit report on request
32
33 ## Data Minimization
34 - Only extract job-relevant information
35 - Auto-mask sensitive fields (ID number, address)
36 - Do not store unnecessary personal data
37 `;

```

G.3 Interview Agent System Prompt

Listing 12: Interview Agent System Prompt

```

1  const INTERVIEW_SYSTEM_PROMPT = `
2  You are HireEasy's Interview Agent, an expert in
3  designing structured interviews for Chinese SMEs.
4
5  ## Your Role
6  Help employers who lack professional interview skills
7  conduct effective, fair, and insightful interviews.
8
9  ## Core Capabilities
10  1. Generate role-specific interview questions
11  2. Create structured interview plans with timing
12  3. Design scoring rubrics for each question
13  4. Generate follow-up questions based on responses
14  5. Produce post-interview evaluation reports
15
16  ## Question Generation Strategy
17  Use the STAR method (Situation-Task-Action-Result):

```

```

18 - Technical Questions: 3-5 based on JD key skills
19 - Behavioral Questions: 2-3 for soft skills
20 - Scenario Questions: 1-2 for problem-solving
21 - Culture Fit Questions: 1-2 for team alignment
22 - Motivation Questions: 1 for career intent
23
24 ## Interview Plan Structure
25 1. Opening (5 min): Introduction, set expectations
26 2. Background Review (10 min): Resume walkthrough
27 3. Technical Assessment (15-20 min): Core skills
28 4. Behavioral Assessment (10-15 min): Soft skills
29 5. Scenario Discussion (10 min): Problem-solving
30 6. Candidate Questions (5-10 min): Q&A
31 7. Closing (5 min): Next steps, timeline
32
33 ## Scoring Rubric Format
34 For each question provide:
35 - Excellent (5): Specific criteria
36 - Good (4): Specific criteria
37 - Average (3): Specific criteria
38 - Below Average (2): Specific criteria
39 - Poor (1): Specific criteria
40 `;

```

G.4 Compliance Agent System Prompt

Listing 13: Compliance Agent System Prompt

```

1  const COMPLIANCE_SYSTEM_PROMPT = `
2  You are HireEasy's Compliance Agent, the guardian of
3  fair and legal recruitment practices.
4
5  ## Your Role
6  Monitor all recruitment activities for legal
7  compliance and ethical AI use, operating as a
8  persistent background checker on every action.
9
10 ## Legal Framework
11 You enforce compliance with:

```

```

12 1. PIPL ( ) - Data protection
13 2. Employment Promotion Law ( ) - Anti-discrimination
14 3. Women's Rights Protection Law
15 4. Disability Protection Law
16 5. EU AI Act (high-risk AI systems) - Transparency
17 6. EEOC Four-Fifths Rule - Statistical fairness
18
19 ## Detection Rules
20 ### JD Bias Detection
21 - Age: "35", " ", " "
22 - Gender: " ", " ", " "
23 - Appearance: " ", " "
24 - Marital: " ", " ", " "
25 - Regional: " ", " XX "
26 - Health: " ", " "
27 - Education snobbery: " 985/211 "
28
29 ### Data Minimization Checks
30 - Flag collection of unnecessary personal data
31 - Ensure consent mechanism exists
32 - Verify data retention policies
33
34 ### Fairness Monitoring
35 - Track pass rates by demographic group
36 - Alert when any ratio < 0.8 (Four-Fifths Rule)
37 - Provide remediation suggestions
38
39 ## Severity Levels
40 - CRITICAL: Active discrimination, legal violation
41 - WARNING: Potentially discriminatory language
42 - INFO: Best practice suggestion
43 `;

```

H 附录 H：数据字典

以下为聘易系统完整数据字典，定义所有枚举值和业务术语。

H.1 状态枚举定义

表 73: 职位状态枚举 (JobStatus)

枚举值	中文名	描述
DRAFT	草稿	JD 正在编辑中，尚未提交审核
REVIEWING	审核中	已提交合规检查，等待 Compliance Agent 审核
PUBLISHED	已发布	通过审核并发布到招聘平台
SCREENING	筛选中	已收到简历，Screen Agent 正在筛选
INTERVIEWING	面试中	已有候选人进入面试环节
CLOSED	已关闭	招聘完成或手动关闭
CANCELLED	已取消	岗位需求取消

表 74: 候选人状态枚举 (CandidateStatus)

枚举值	中文名	描述
NEW	新简历	简历刚收到，尚未处理
PARSED	已解析	简历已完成结构化解析
SCORED	已评分	AI 已完成匹配评分
SHORTLISTED	入围	进入候选短名单
INTERVIEWING	面试中	已安排面试
INTERVIEW_PASSED	面试通过	面试评估通过
OFFERED	已发 Offer	已向候选人发出录用通知
HIRED	已录用	候选人接受 Offer 并入职
REJECTED	已淘汰	任何环节被淘汰
WITHDRAWN	已放弃	候选人主动退出

表 75: 合规严重级别枚举 (ComplianceSeverity)

枚举值	中文名	描述
CRITICAL	严重	违反法律法规，必须立即修复，阻止发布
WARNING	警告	可能存在歧视风险，建议修改
INFO	建议	最佳实践建议，不影响发布

H.2 行业分类编码

表 76: 支持行业分类

编码	行业	包含子行业
IND_FOOD	餐饮服务	快餐、正餐、奶茶/咖啡、外卖
IND_ECOM	电商零售	跨境电商、直播电商、社区团购
IND_MANU	制造业	精密制造、电子元件、食品加工
IND_EDUC	教育培训	K12、职业培训、语言培训、在线教育
IND_TECH	科技互联网	SaaS、移动开发、人工智能、大数据
IND_MEDI	医疗健康	诊所、药房、健康管理
IND_SERV	商务服务	代账、法律、咨询、广告
IND_CONS	建筑工程	装修、施工、设计
IND_LOGI	物流运输	快递、仓储、配送
IND_OTHE	其他	未分类行业

H.3 技能标签体系

表 77: 技术技能标签分类（部分）

技能类别	标签示例
编程语言	Python, JavaScript, TypeScript, Java, Go, Rust, C++, PHP
前端框架	React, Vue, Angular, Next.js, Svelte, Tailwind CSS
后端框架	Django, Flask, Express, Spring Boot, Gin, FastAPI
数据库	PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch
云服务	AWS, 阿里云, 腾讯云, GCP, Azure, K8s, Docker
AI/ML	PyTorch, TensorFlow, Scikit-learn, LangChain, RAG
设计工具	Figma, Sketch, Photoshop, Illustrator, Canva
运营技能	SEO, SEM, 数据分析, 用户运营, 内容运营
管理技能	项目管理, 团队管理, 战略规划, 绩效管理

I 附录 I：术语表

表 78: 聘易系统核心术语表

术语	定义
Agent	具备自主决策能力的 AI 智能体，能使用工具完成特定任务
Harness	Agent 的运行时框架，包含工具管理、权限控制、上下文管理等
JD	Job Description，职位描述
ATS	Applicant Tracking System，候选人跟踪系统
PIPL	Personal Information Protection Law，《个人信息保护法》
EU AI Act	欧盟人工智能法案
EEOC	Equal Employment Opportunity Commission，平等就业机会委员会
80% 规则	四分之五规则，评估招聘甄选程序公平性的统计标准
PLG	Product-Led Growth，产品驱动增长
SaaS	Software as a Service，软件即服务
TAM	Total Addressable Market，总可用市场
SAM	Serviceable Available Market，可服务市场
SOM	Serviceable Obtainable Market，可获市场
CAC	Customer Acquisition Cost，客户获取成本
NPS	Net Promoter Score，净推荐值
MAU	Monthly Active Users，月活跃用户
STAR	Situation-Task-Action-Result，行为面试法
RBAC	Role-Based Access Control，基于角色的访问控制
RAG	Retrieval-Augmented Generation，检索增强生成
Prompt 注入	通过恶意输入操控 AI 行为的攻击方式
AI 幻觉	AI 生成看似正确但实际不准确或虚构的内容
数据飞轮	更多用户 → 更多数据 → 更好模型 → 更好体验的正反馈循环

J 附录 G：行业应用案例

J.1 案例 1：教育科技公司

星辰教育科技 · 50 人 · K12 在线教育

背景：公司完成 A 轮融资后快速扩张，需在 1 个月内招聘 5 名技术人员（前端 2 名、后端 2 名、测试 1 名），但只有 1 名行政人员兼任 HR。

痛点：

- 行政人员不了解技术岗位要求，写的 JD 千篇一律
- 每个岗位收到 50+ 份简历，但 90% 不相关
- CEO 亲自面试，每天只能面 2 人，严重影响业务
- 曾因 JD 中写”限男性”被应聘者投诉到劳动监察

使用聘易的完整流程：

1. **建立企业档案**（层 1）：录入公司信息、技术栈偏好、薪资策略
2. **JD Agent**：一键生成 5 个技术岗 JD，每份含职责、要求、福利和团队介绍
3. **JD 诊断**：发现后端 JD 的 SEO 评分仅 55 分，优化后提升至 82 分
4. **Compliance Agent**：自动扫描新旧 JD，清除所有歧视性表述
5. **Screen Agent**：对 245 份简历批量筛选，推荐面试 33 人
6. **Interview Agent**：为每位候选人生成针对性面试方案
7. **记忆系统**：自动总结”教育科技行业技术岗招聘最佳实践”

量化效果：

指标	使用前	使用后
JD 撰写时间	2 小时/份	15 分钟/份
简历筛选时间	4 小时/岗	30 分钟/岗
平均招聘周期	45 天	18 天
JD 诊断评分	55 分	85 分
合规风险	3 项	0 项

J.2 案例 2：餐饮连锁企业

味来餐饮 · 20 人总部 + 8 家门店 · 餐饮连锁

背景：快速扩张中的餐饮连锁，每月需要招聘 15-20 名门店员工（服务员、厨师、店长），创始人亲自管理招聘。

痛点：

- JD 极其简单：“招服务员，月薪 3000-5000，限女性”
- 不了解就业法规，多处存在合规风险
- 高流失率——试用期离职率 40%
- 面试无标准，凭第一印象决定

使用聘易后：

1. Compliance Agent 扫描 8 个岗位 JD，发现 12 项合规风险
2. JD Agent 针对餐饮行业生成标准化 JD 模板，重点突出薪资透明、晋升通道、员工餐等福利
3. Interview Agent 生成服务员面试标准化问题集（服务意识、抗压能力、团队协作）
4. 层 4 记忆系统积累”餐饮行业高留存员工特征”：稳定居住地 + 上一份工作 ≥6 个月

量化效果：合规风险从 12 项降至 0 项，投递量提升 120%，试用期离职率从 40% 降至 15%。

J.3 案例 3：软件外包公司

码力科技 · 30 人 · 软件外包/技术咨询

背景：项目制运营，需要快速组建不同技术栈的团队。项目周期 3-6 个月，人员需求变化频繁。

痛点：

- 每个新项目都要从零写 JD，效率极低
- 需要同时招 Java/Python/前端/移动端等多种技术栈
- 经常”救火式”招聘，质量难保障
- HR 对技术不了解，无法准确评估候选人

使用聘易后：JD Agent 积累了各技术栈的 JD 模板库，新项目 JD 生成时间从 2 天缩短到 20 分钟。Screen Agent 建立多技术栈匹配模型，HR 不需要了解技术细节也能完成初筛。

量化效果：新项目组队时间从 3 周缩短到 1 周，人岗匹配率从 65% 提升到 88%。

J.4 案例 4：跨境电商

海阔电商 · 40 人 · 跨境电商

背景：东南亚市场电商公司，需要同时招聘国内运营、海外 BD、物流等岗位，涉及中英文双语。

使用聘易后：JD Agent 生成中英文双语 JD，Compliance Agent 分别按中国《就业促进法》和目标市场法律检查合规性，Screen Agent 对跨语言简历进行标准化评分。

量化效果：跨境岗位招聘周期缩短 50%，双语 JD 合规率 100%，海外人才库建立效率提升 3 倍。

J.5 案例 5：医疗健康初创

康未来科技 · 12 人 · 数字医疗

背景：医疗 AI 初创公司，需招聘 AI 工程师和医学顾问，对数据安全合规要求极高。

特殊要求：候选人医疗执业资格信息属于《个人信息保护法》第 28 条定义的”敏感个人信息”，必须取得单独同意并加密存储。

使用聘易后：Compliance Agent 自动识别敏感数据收集场景，生成合规提示和隐私声明模板。所有候选人简历按数据分级策略（L1-L4）管理，医疗执照信息启用 L4 级加密。

量化效果：通过医疗行业数据安全审计，招聘流程合规率从 60% 提升到 98%。

K 附录 H：数据安全架构详细设计

K.1 数据分级保护策略

表 79: 候选人数据分级与保护措施

等级	分类	数据类型	保护措施
L1	公开	岗位名称、JD 内容、公司介绍	无特殊要求
L2	内部	候选人姓名、联系方式、教育背景	访问控制 + 脱敏展示
L3	机密	简历全文、面试记录、评分	AES-256 加密 + RBAC + 审计日志
L4	绝密	身份证号、薪资信息、医疗/犯罪记录	端到端加密 + 单独同意 + 90 天定期销毁

K.2 数据全生命周期管理



图 21: 候选人数据全生命周期管理流程

K.3 API 安全设计

表 80: API 安全控制措施

安全层	实现方案
传输加密	HTTPS/TLS 1.3, HSTS 强制, 证书钉扎
身份认证	JWT Token (15 分钟有效期) + Refresh Token (7 天) 双令牌
权限控制	RBAC 三级权限 (管理员/HR/只读), 资源级细粒度控制
请求限流	令牌桶算法, 普通用户 60 req/min, 付费用户 300 req/min
输入校验	zod schema 验证 + XSS 过滤 + SQL 注入防护 + prompt 注入防护
操作审计	全操作日志 (who/when/what), 保留 365 天, 不可篡改
密钥管理	环境变量隔离, HashiCorp Vault 托管, 90 天自动轮换
异常监控	实时告警 (连续失败认证 >5 次 / 异常数据访问模式)

L 附录 I: 80% 规则实施细则

L.1 监控指标定义

表 81: 80% 规则各环节监控指标

环节	定义	计算方式	预警阈值
简历通过率	通过初筛的简历占该群体总投递的比例	$\frac{\text{群体通过数}}{\text{群体投递数}}$	< 80%
面试通过率	通过面试的候选人占该群体面试总数的比例	$\frac{\text{群体面试通过}}{\text{群体面试总数}}$	< 80%
Offer 率	收到 Offer 的候选人占该群体面试通过总数的比例	$\frac{\text{群体 Offer 数}}{\text{群体面试通过数}}$	< 80%
整体录用率	最终录用占该群体总投递的比例	$\frac{\text{群体录用数}}{\text{群体投递数}}$	< 80%

L.2 实施示例

场景：某公司”前端开发”岗位收到 100 份简历，其中男性 70 份、女性 30 份。
筛选结果：

- 男性通过初筛：28 人（通过率 = $28/70 = 40\%$ ）
- 女性通过初筛：6 人（通过率 = $6/30 = 20\%$ ）

80% 规则计算：

$$\text{影响比} = \frac{\text{女性通过率}}{\text{男性通过率}} = \frac{20\%}{40\%} = 50\% < 80\%$$

聘易 Compliance Agent 反应：

1. 自动预警：”前端开发岗位简历筛选环节性别通过率比为 50%，低于 80% 阈值”
2. 生成分析报告：列出被拒女性候选人的拒绝原因分布
3. 建议改进：检查筛选标准是否包含与岗位无关的性别关联因素
4. 合规记录：自动记录事件到审计日志，供合规审查使用

M 附录 J: 术语表

术语	定义
Agent	能够自主执行任务的 AI 智能体，由模型（Model）和工程框架（Harness）组成
Harness	Agent 的工程”骨架”，包括编排引擎、工具系统、记忆管理、安全治理等基础设施
JD（Job Description）	职位描述，包含岗位职责、任职要求、薪资福利等信息
ReAct 循环	Reasoning + Acting，Agent 交替推理和执行工具调用的运行模式
System Prompt	系统提示词，定义 Agent 的角色、能力边界和行为规范
Tool Protocol	工具协议，每个工具的输入输出 schema、权限要求、并发控制规则
PIPL	Personal Information Protection Law，《中华人民共和国个人信息保护法》
EU AI Act	欧盟人工智能法案，全球首部综合性 AI 监管法规
80% 规则	EEOC 的四分之五规则，用于判断招聘筛选是否存在群体歧视
RBAC	Role-Based Access Control，基于角色的访问控制
Token	AI 模型处理的最小文本单位，中文约 1 个字 =1.5-2 个 token
Few-shot	少样本提示，在 prompt 中提供几个示例帮助模型理解任务
Zustand	React 轻量级状态管理库，支持持久化中间件
TTS	Text-to-Speech，文本转语音技术
ATS	Applicant Tracking System，简历追踪系统
NPS	Net Promoter Score，净推荐值，衡量用户满意度
SaaS	Software as a Service，软件即服务
MCP	Model Context Protocol，模型上下文协议